

A Project report on

**DEEPAKE DETECTION FOR IDENTIFYING MACHINE
GENERATED TWEETS USING NLP AND DL**

*Submitted in partial fulfillment of the requirements
for the award of the degree of*

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING (DATA SCIENCE)

By

EDIGA MADHURI	214G1A3250
AVULAPALLI NADEEM	214G1A3257
DAYYAM CHARAN KUMAR REDDY	214G1A3212
YERUVA HARSHAVARDHAN REDDY	224G5A3206

Under the Guidance of

Mrs. G. Shabana M.Tech.,
Assistant Professor



Computer Science and Engineering (Data Science)

**SRINIVASA RAMANUJAN INSTITUTE OF TECHNOLOGY
(AUTONOMOUS)**

Rotarypuram Village, B K Samudram Mandal, Ananthapuramu - 515701

2024-2025

SRINIVASA RAMANUJAN INSTITUTE OF TECHNOLOGY

(AUTONOMOUS)

(Affiliated to JNTUA, Accredited by NAAC with 'A' Grade, Approved by AICTE, New Delhi & Accredited by NBA (EEE, ECE & CSE))

Rotarypuram Village, BK Samudram Mandal, Ananthapuramu-515701

COMPUTER SCIENCE AND ENGINEERING (DATA SCIENCE)



Certificate

This is to certify that the project report entitled **Deepfake detection for Identifying Machine Generated Tweets using NLP and DL** is the bonafide work carried out by **Madhuri E, Nadeem A, Charan Kumar Reddy D, Harshavardhan Lourdu Reddy Y** bearing Roll Number **214G1A3250, 214G1A3257, 214G1A3212, 224G5A3206** in partial fulfilment of the requirements for the award of the degree of **Bachelor of Technology** in **Computer Science and Engineering (Data Science)** during the academic year 2024-2025.

Project Guide

Mrs. G. Shabana, M.Tech.,
Assistant Professor

Head of the Department

Dr. P. Chitralingappa, M.Tech., Ph.D.,
Associate Professor

Date:

Place: Rotarypuram

External Examiner

DECLARATION CERTIFICATE

We students of **Computer Science and Engineering (Data Science)**, **SRINIVASA RAMANUJAN INSTITUTE OF TECHNOLOGY(AUTONOMOUS)**, Rotarypuram, hereby declare that the dissertation entitled **DEEPFAKE DETECTION FOR IDENTIFYING MACHINE GENERATED TWEETS USING NLP AND DL** embodies the report of our project work carried out by us during IV year under the guidance of **Mrs. G. Shabana**, Assistant Professor, Department of Computer Science and Engineering (AI&ML), Srinivasa Ramanujan Institute of Technology, and this work has been submitted for the partial fulfillment of the requirements for the award of degree of Bachelor of Technology.

The results embodied in this project report have not been submitted to any other University or Institute for the award of any Degree or Diploma.

Date:

Place: Anantapur

S. No.	Name of the Student	Roll Number	Signature
1	E. Madhuri	214G1A3250	
2	A. Nadeem	214G1A3257	
3	D. Charan Kumar Reddy	214G1A3212	
4	Y. Harshavardhan Lourdu Reddy	224G5A3206	

Vision & Mission of the SRIT

Vision:

To become a premier Educational Institution in India offering the best teaching and learning environment for our students that will enable them to become complete individuals with professional competency, human touch, ethical values, service motto, and a strong sense of responsibility towards environment and society at large.

Mission:

- Continually enhance the quality of physical infrastructure and human resources to evolve in to a center of excellence in engineering education.
- Provide comprehensive learning experiences that are conducive for the students to acquire professional competences, ethical values, life-long learning abilities and understanding of the technology, environment and society.
- Strengthen industry institute interactions to enable the students work on realistic problems and acquire the ability to face the ever changing requirements of the industry.
- Continually enhance the quality of the relationship between students and faculty which is a key to the development of an exciting and rewarding learning environment in the college.

Vision & Mission of the Department of CSE (Data Science)

Vision:

To evolve as a leading department by offering best comprehensive teaching and learning practices for students to be self-competent technocrats with professional ethics and social responsibilities.

Mission:

- DM 1: Continuous enhancement of the teaching-learning practices to gain profound knowledge in theoretical & practical aspects of computer science applications.
- DM 2: Administer training on emerging technologies and motivate the students to inculcate self-learning abilities, ethical values and social consciousness to become competent professionals.
- DM 3: Perpetual elevation of Industry-Institute interactions to facilitate the students to work on real-time problems to serve the needs of the society.

Program Educational Objectives (PEOs)

An SRIT graduate in Computer Science & Engineering (Data Science), after three to four years of graduation will:

- PEO 1: Lead a successful professional career in IT / ITES industry / Government organizations with ethical values.
- PEO 2: Become competent and responsible computer science professional with good communication skills and leadership qualities to respond and contribute significantly for the benefit of society at large.
- PEO 3: Engage in life-long learning, acquiring new and relevant professional competencies / higher academic qualifications.

ACKNOWLEDGEMENT

The satisfaction and euphoria that accompany the successful completion of any task would be incomplete without the mention of people who made it possible, whose constant guidance and encouragement crowned our efforts with success. It is a pleasant aspect that we have now the opportunity to express our gratitude for all of them.

It is with immense pleasure that we would like to express our indebted gratitude to our Guide **Mrs. G. Shabana, Assistant Professor, Computer Science and Engineering (AI&ML)**, who has guided us a lot and encouraged us in every step of the project work. We thank her for the stimulating guidance, constant encouragement and constructive criticism which have made possible to bring out this project work.

We are very much thankful to **Dr. P. Chitralingappa, Associate Professor & Head of the Department, Computer Science and Engineering (Data Science)**, for his kind support and for providing necessary facilities to carry out the work.

We wish to convey our special thanks to **Dr. G. Balakrishna, Principal of Srinivasa Ramanujan Institute of Technology**, for giving the required information in doing our project work. Not to forget, we thank all other faculty and non-teaching staff, and our friends who had directly or indirectly helped and supported us in completing our project in time.

We also express our sincere thanks to the Management for providing excellent facilities. Finally, we wish to convey our gratitude to our family who fostered all the requirements and facilities that we need.

Project Associates

214G1A3250

214G1A3257

214G1A3212

224G5A3206

ABSTRACT

Recent advancements in natural language generation have provided powerful tools capable of manipulating public opinion on social media platforms. The proliferation of text-generative models has enabled adversaries to enhance social bots, generating realistic deepfake posts that influence public discourse. This research addresses the pressing issue of identifying machine-generated text, focusing on social networks like Twitter. The primary objective of this study is to develop reliable and accurate methods for detecting deepfake social media messages.

To achieve this goal, we propose a deep learning-based approach employing a Convolutional Neural Network (CNN) architecture combined with FastText word embeddings to classify tweets as either human-generated or bot-generated. The research utilized the publicly available Tweepfake dataset and compared the performance of the proposed method against several baseline machine learning models. These baselines incorporated features such as Term Frequency (TF), Term Frequency-Inverse Document Frequency (TF-IDF), FastText, and FastText subword embeddings.

The results of the study demonstrated the superior performance of the proposed CNN model with FastText embeddings, highlighting its capability to accurately detect deepfake tweets. These findings underscore the importance of leveraging deep learning and advanced word embeddings for tackling the growing challenge of machine-generated content. This research contributes to the development of robust tools for safeguarding public discourse on social media, with significant implications for combating misinformation and ensuring the integrity of online interactions.

Keywords: *Deepfake detection, Convolutional Neural Network (CNN), Natural Language Processing (NLP), Machine-generated tweets, FastText embeddings.*

Contents

List of Figures	ix
List of Tables	x
List of Abbreviations	xi
Nomenclatures	xii
Chapter 1: Introduction	1-2
1.1 Problem Statement	1
1.2 Objectives	2
1.3 Scope of the study	2
1.4 Significance of the study	2
Chapter 2: Literature Survey	3-4
Chapter 3: Planning	5-12
3.1 Existing System	5
3.1.1 Disadvantages	6
3.2 Proposed System	7
3.2.1 Advantages	7
3.3 Hardware Requirements	8
3.4 Software Requirements	8
3.4.1 Software Dependencies	9
3.4.2 Library Requirements	9
3.5 Functional Requirements	10
3.6 Non-Functional Requirements	11
3.7 Objectives of planning	11
3.7.1 Performance	12
3.7.2 Cost	12
3.7.3 Time	12
Chapter 4: System Design	13-19
4.1 System Architecture	13
4.1.1 Data flow diagram	13

4.1.2	UML diagram	16
4.1.3	Class diagram	16
4.1.4	Activity diagram	17
4.1.5	Sequence diagram	17
4.1.6	Collaboration diagram	18
4.1.7	Component diagram	19
Chapter 5:	Implementation	20-52
5.1	Implementation setup: Python Installation	20
5.2	Implementation setup: Environment files	20
5.2.1	Dependency file (set1.txt)	21
5.2.2	NLP Dataset Downloader file (nltkdownload.py)	21
5.2.3	Django Project Management file (manage.py)	22
5.3	Implementation setup: Django Environment	23
5.4	Implementation: CNN Model	24
5.4.1	Prediction Flow in views.py	26
5.5	Implementation: Backend (views.py)	27
5.6	Implementation: Template – View Mapping (urls.py)	35
5.7	Implementation: Frontend Web Interface	37
5.7.1	Template Interactions with Views	50
5.8	Deepfake Project Directory Structure	52
Chapter 6:	Testing	53-55
6.1	Features to be Tested	53
6.2	Testing tools and Environment	53
6.3	Testing Procedure	54
6.4	Inputs and Expected outputs	55
Chapter 7:	Results	56-62
Conclusion		63
References		64
Publication		65-69

List of Figures

Fig. No	Description	Page No
4.1	System Architecture	13
4.2	Data Flow Diagram	15
4.3	UML diagram	16
4.4	Class Diagram	16
4.5	Activity Diagram	17
4.6	Sequence Diagram	18
4.7	Collaboration Diagram	18
4.8	Component Diagram	19
5.1	Workflow mapping of URLs, views, Templates	51
5.2	Project Directory Structure	52
7.1	Python Server Launch in Console	56
7.2	Welcome Page	56
7.3	Login Page	57
7.4	User Registration page	57
7.5	Existing User Login Page	58
7.6	User Home Page	58
7.7	Admin Login	59
7.8	Users Table	59
7.9	Application Home Page	60
7.10	Performance Graphs	60
7.11	Deepfake Detection page	61
7.12	Output as Normal Tweet	61
7.13	Output as Bot Generated Tweet	62

List of Tables

Table No	Table Name	Page No
3.1	Existing System Disadvantages	6
3.2	Proposed System Advantages	7
3.3	List of Libraries	9
5.1	Template – View Mapping	50
6.1	Expected Outputs for each Testcase	55

LIST OF ABBREVIATIONS

ML	Machine Learning
DL	Deep Learning
NLP	Natural Language Processing
AI	Artificial Intelligence
TF-IDF	Term Frequency-Inverse Document Frequency
TF	Term Frequency
CNN	Convolutional Neural Network
RF	Random Forest
NB	Naïve Bayes
LR	Logistic Regression
DT	Decision Tree
GB	Gradient Boosting
Bi - LSTM	Bidirectional Long Short-Term Memory
CSV	Comma-Separated Values
HDF5	Hierarchical Data Format version 5
pckl	Pickle format

NOMENCLATURE

$TF(t)$	Term Frequency
$IDF(t)$	Inverse Document Frequency
$TF-IDF(t)$	TF-IDF Score
α	Learning rate
L	Loss Function
W	Model weights
$\sigma(z_i)$	SoftMax Function
y_i	True label
$\sum$	Summation

CHAPTER 1

INTRODUCTION

The four main forms of user-generated content on social media platforms are text, images, audio, and video. Bots utilize techniques like deepfake, search-and-replace, and gap-filling text to operate phony social media accounts, engage with posts, and publish content. Deep learning enables the analysis of incoming data to learn feature representation, allowing AI-generated text, images, audio, and video to appear convincingly real. Deepfake multimedia, by mimicking human-generated content, has created challenges in various fields, particularly in politics. Misinformation spreads rapidly on social media, manipulating public opinion through automated bots and fake accounts. Advanced AI models like Grover and GPT generate convincing fake content, making detection difficult. During the 2024 U.S. elections, AI-generated misinformation influenced public discourse and caused confusion. Fabricated news articles highlighted the growing challenge of distinguishing reality from fake content.

The rise of complex transformer-based models has further complicated the detection of misinformation. These models automate the generation of blog posts, tweets, and online discussions, demonstrating their ability to create human-like text. One example involved a GPT-3 bot responding to queries on an online forum under a human-like username, where its responses were largely innocuous. To protect genuine information and uphold democratic integrity on social media, it is crucial to develop sovereign systems capable of detecting deepfake texts and other forms of machine-generated misinformation.

1.1 Problem Statement

The integrity of information shared on social media platforms is at risk because of the fast development of natural language processing algorithms, which have enabled increasingly lifelike machine-generated text. With the use of sophisticated text-generative models, adversaries can construct deepfake posts that sway public opinion and conversation. Despite growing concerns, there are currently no effective methods for identifying tweets that are generated by machines among the vast amount of user-generated content.

1.2 Objectives

The study seeks to enhance the reliability of online interactions.

- i. **To Develop and evaluate a deep learning model** - Build an AI driven system using FastText embeddings to classify human and machine-generated tweets.
- ii. **Analyze linguistic and contextual features** - Examine the unique text characteristics of AI-generated tweets for effective detection.
- iii. **Improve misinformation detection strategies** - Provide insights that help in the development of tools for combating fake content on social media platforms.

1.3 Scope of the Study

This study focuses on the detection of AI-generated misinformation on social media, specifically machine-generated tweets. It examines linguistic features and contextual embeddings using FastText to distinguish between human-authored and AI-generated content. The study is limited to analyzing text-based social media deepfakes and does not cover other forms of deepfake media such as images, audio, or video.

- Text-based deepfake detection
- Use of FastText embeddings
- Implementation of CNN for text classification

1.4 Significance of the Study

This study addresses the growing challenge of detecting AI-generated misinformation on social media platforms. By developing a deep learning model using FastText embeddings, the study contributes to efforts in combating deep-fake content and improving the reliability of online interactions. The findings aid researchers and social media platforms in implementing more effective misinformation detection strategies.

- Enhancing misinformation detection
- Improving trust in digital interaction
- Contributing to AI ethics and policy development

CHAPTER 2

LITERATURE SURVEY

A **literature survey** is a detailed and organized summary of all the significant research and studies that have been conducted on a particular topic. It involves collecting, analyzing, and evaluating scholarly articles, books, journals, and other academic sources related to the subject of interest. The main goal of a literature survey is to understand the current state of knowledge in the field, identify trends and patterns, recognize gaps or limitations in existing research, and establish a foundation for new studies. It helps researchers avoid repeating past work and supports the development of new ideas by building on existing findings. In a research project or thesis, the literature survey serves as a crucial component that justifies the need for the study and shows how it fits into the broader academic context.

[1]. T. Fagni, F. Falchi, M. Gambini, et al., “Tweepfake: About detecting deepfake tweets,” PLOS One, vol. 16, pp. 1–20, Jan. 2021.

Explored the detection of deepfake tweets using machine learning models, with a primary focus on BERT-based classifiers. Their study aimed to distinguish between AI-generated and human-authored tweets by leveraging transformer models to analyze linguistic patterns. The researchers trained their model on datasets containing both human and machine-generated content, achieving high accuracy in classification. The findings demonstrated that AI-generated tweets often exhibit specific textual characteristics that enable effective detection. However, a notable limitation was the dataset bias, as the study relied on a limited sample of tweets, which may affect the generalizability of the model. Additionally, as generative AI continues to advance, deepfake detection methods must evolve to counter increasingly sophisticated AI-generated content.

[2]. S. Mitrović, D. Andreoletti, and O. Ayoub, “ChatGPT or human? Detect and explain,” arxiv preprint, vol. 2023, pp. 1–10, Jan. 2023.

They investigated the ability to distinguish ChatGPT-generated text from human-written content using transformer-based models. Their study focused on both classification and interpretability, aiming to explain why specific text outputs were classified as AI-generated. The researchers identified **linguistic and stylistic markers**

that differentiated AI text from human writing, such as excessive syntactic consistency and overuse of certain phrases. However, the study noted that the effectiveness of detection methods was **heavily model-dependent**, meaning newer, fine-tuned generative models might bypass existing detection techniques. Additionally, adversarial attacks—where minor modifications in AI-generated text could fool detection models—remained a significant challenge.

[3]. R. Vaibhav, R. Furqan, A. Aashir, P. B. Washington, E. Lee, and A. Imran, “Deepfake tweets classification using stacked Bi-LSTM and word embeddings,” PeerJ Comput. Sci., vol. 7, pp. e745, Dec. 2021.

Proposed a deep learning approach using stacked Bi-LSTM (Bidirectional Long Short-Term Memory) networks combined with word embeddings to classify deepfake tweets. The study demonstrated that Bi-LSTM models were effective in capturing contextual dependencies, leading to the generation of human-like tweets with low perplexity. The authors highlighted that their approach enhanced the diversity and authenticity of machine-generated tweets, making them more difficult to distinguish from human-authored content. Despite its effectiveness, the model faced limitations in handling sarcasm and ambiguous text, which are common in social media communication. Additionally, Bi-LSTM models are computationally expensive, making them less efficient compared to transformer-based alternatives such as BERT or GPT-based architectures.

[4]. A. Heidari, N. J. Navimipour, H. Dag, and M. Unal, “Deepfake detection using deep learning methods: A systematic and comprehensive review,” WIREs Data Mining and Knowledge Discovery, vol. 14, no. 2, pp. e1520, Mar. 2024.

Conducted a systematic review of deepfake detection methods across multiple domains, including text, image, audio, and video deepfakes. Their study analyzed the effectiveness of deep learning-based approaches such as CNNs (Convolutional Neural Networks), GANs (Generative Adversarial Networks), and transformers in identifying synthetic content. CNNs were found to be highly effective for image-based deepfake detection, while transformer-based models like BERT and GPT proved useful for text deepfake detection. GANs played a dual role—both as a tool for generating realistic deepfakes and as an adversarial approach for detection.

CHAPTER 3

PLANNING

3.1 Existing System

The detection of deepfake text and multimedia has been a growing area of research, with various machine learning and deep learning approaches being explored. Early methods primarily relied on feature-based and graph-based models to analyze textual patterns, but with the rise of advanced generative models, deep learning techniques such as BiLSTM and transformer-based architectures have become more prominent. Studies like Tweepfake have demonstrated that models like BERT can effectively classify AI-generated tweets with high accuracy. Similarly, research on ChatGPT-generated content has highlighted the importance of explainability, showing that specific linguistic and stylistic features can be used to differentiate human-written text from AI-generated text. However, these approaches remain limited by dataset diversity and the adaptability of newer generative models. In addition to text-based detection, studies analyzing deepfake political videos have shown that public awareness of deepfakes may lead to increased skepticism toward all media content, whether real or fake. This suggests that detection models alone are insufficient; there is a need for an integrated framework that combines technical detection with media literacy initiatives. While this approach improves fluency and realism in generated content, it struggles with detecting sarcasm and ambiguous language, which are prevalent in social media communication.

A comprehensive review of deepfake detection techniques has identified CNNs, GANs, and transformer-based models as key tools for tackling deepfake content across various domains. CNNs have proven effective for image-based deepfake detection, while transformers like BERT and GPT have shown promise in identifying text-based deepfakes. Despite these advancements, there is no unified framework for deepfake detection, making it difficult to apply these models effectively on a large scale. Future research must focus on developing scalable, explainable, and robust deepfake detection systems that can adapt to rapidly evolving AI-generated content.

3.1.1 Disadvantages

Challenge	Description
Dataset bias	Many existing studies rely on specific datasets that lack diversity in language, context, and style, leading to biased models that struggle with real-world generalization.
Lack of robustness	Detection methods are often model-dependent and may fail to identify deepfakes generated by fine-tuned AI models.
Sarcasm and ambiguity detection issues	Models such as Bi-LSTM struggle with understanding sarcasm, and ambiguous statements, which are common in social media posts.
High computational cost	Some deep learning models, like Bi-LSTM and transformer-based architectures, require significant computational resources, making them impractical for large-scale detection.
Absence of a unified framework	There is no standardized deepfake detection approach that integrates multiple techniques into a single, scalable system for text.
Increased public distrust	Studies have shown that simply warning users about deepfakes can lead to skepticism toward all media content rather than improving their ability to discern fake from real.

Table 3.1: Existing System Disadvantages

3.2 Proposed System

The proposed system enhances deepfake text detection by leveraging FastText embeddings and optimized deep learning architectures, specifically a CNN-based model. With the rise of deepfake text, distinguishing between human-written and machine-generated content has become increasingly challenging. Our system tackles this issue using FastText embeddings and a CNN-based deep learning model, making deepfake detection more accurate, efficient, and scalable. However, FastText embeddings break words into smaller subunits, allowing the system to better understand word variations, context, and nuanced meanings. This is particularly useful on social media platforms where abbreviations, slang, and informal language are common. Unlike BERT and RoBERTa, which require extensive fine-tuning and high computational power, our CNN-based approach is lightweight yet highly effective, making it accessible to users with limited resources.

Additionally, our system is designed for real-time detection, which is crucial in today's fast-paced digital world. Imagine a scenario where a viral tweet falsely claims that a political leader has resigned, leading to mass confusion. With our proposed approach, the system can quickly flag such deepfake content before it spreads widely. The model's efficiency also allows for deployment on large-scale social media datasets, making it an ideal solution for platforms like Twitter and Facebook that deal with massive amounts of user-generated content daily. Beyond detection, this research lays the groundwork for future advancements in AI-driven misinformation control. By addressing the limitations of existing systems, our approach not only helps in filtering out deepfake content but also encourages the responsible development of AI-generated text models.

3.2.1 Advantages

Challenge	Description
High precision	Achieves superior accuracy in distinguishing deepfake text from human-authored content.

Better handling of sarcasm and ambiguity	FastText embeddings improve interpretation of complex and informal language.
Real-time detection	Capable of identifying deepfake text as it appears on social media.
Scalability	Can be applied to large datasets, making it practical for social media monitoring.
Cost-effectiveness	Avoids the heavy computational costs associated with transformer-based models.
Better misinformation control	Helps prevent the rapid spread of false information online.

Table 3.2: Proposed System Advantages

3.3 Hardware Requirements

1. **Processor:** Intel Core i5 / AMD Ryzen 5 or above
2. **RAM:** 8 GB and above
3. **Storage:** 25 GB in local drive
4. **Operating System:** Windows 7 or above
5. **Network:** Stable broadband internet connection

3.4 Software Requirements

1. Primary Language: Python
2. Frontend Framework: Django
3. Frontend Technologies: HTML, CSS, JavaScript and Bootstrap4
4. Tools: Jupyter Notebook / VS Code

3.4.1 Software Dependencies

The following software and libraries are essential for the system's functioning:

- Python Version: Python 3.7.0

Ensure "Add to PATH" is checked during installation.

- Django Framework: Django 2.1.7

Used for creating web interface for deepfake text detection.

- NLTK (Natural Language Toolkit):

Install using `download_nltk.bat`

3.4.2 Library Requirements

The following Python libraries are required for data processing, deep learning model training, and visualization:

Library	Version	Purpose
numpy	1.19.2	Numerical computations and matrix operations
pandas	0.25.3	Data manipulation and analysis
Django	2.1.7	Web framework for building the UI
scikit-learn	1.0.2	ML utilities for text classification
nltk	3.4.5	NLP for text analysis
Matplotlib	3.1.1	Data visualization
keras	2.3.1	Deep learning framework for building CNN models

tensorflow	1.14.0	Backend for deep learning model execution
h5py	2.10.0	Managing deep learning model storage in HDF5 format
protobuf	3.16.0	Serialization format used in TensorFlow models

Table 3.3: List of Libraries

3.5 Functional Requirements:

The system consists of the following key functional modules:

1. Data Collection Module

- Collects real and machine-generated tweets for model training.
- Ensures data integrity by removing duplicates and corrupted entries.

2. Data Pre-processing Module

- Cleans and formats the collected text data.
- Performs tokenization, stopwords removal, and stemming or lemmatization.
- Implements FastText embeddings for better word representation.

3. Model Training and Testing Module

- Trains the CNN-based deep learning model using labeled datasets.
- Splits data into training and testing sets for model validation.
- Evaluates model performance using accuracy, precision, and recall metrics.

4. Modelling Module

- Deploys the trained CNN model within the Django framework.
- Integrates an interactive web interface for user interaction.
- Ensures model optimization for real-time deepfake text detection.

5. Prediction Module

- Allows users to input text and get predictions on whether it's human or AI-generated.
- Displays confidence scores and explanations for model decisions.

3.6 Non-Functional Requirements:

One approach of gauging software quality is via Non-Functional Requirement (NFR) evaluations. Responsivity, usability, security, and portability are important but non-functional aspects that decide the success of a software system. Something that isn't working is what the question "how fast does the website load?" is describing. So, both practical and non-practical requirements must be specified.

1. **Usability Requirement** – The system should have an intuitive user interface that is easy to navigate, ensuring a smooth experience for users with minimal training.
2. **Security Requirement** – The project should implement basic security measures such as authentication and data encryption to protect against unauthorized access.
3. **Scalability Requirement** – Although a small-scale project, the system should be designed to accommodate potential growth in users and data without significant rework.
4. **Maintainability Requirement** – The system should be easy to update and modify, allowing developers to fix bugs and add features with minimal effort.
5. **Reliability Requirement** – The software should function consistently under normal conditions, ensuring stable performance without frequent crashes or failures.

3.7 Objectives of Planning

Effective planning is crucial for the successful execution of any project. In this project, planning ensures that the system is developed efficiently while meeting performance, cost, and time constraints. The objectives of planning include optimizing resources and achieving a balance between system functionality and feasibility.

3.7.1 Performance

The system is designed to handle large volumes of social media text efficiently, ensuring quick response times and high accuracy. Optimized deep learning models, such as CNN with FastText embeddings, are implemented to enhance computational efficiency. Performance considerations also include the system's ability to handle linguistic diversity, sarcasm, and ambiguity while maintaining reliability under different workloads.

3.7.2 Cost

The system is designed to minimize computational resource requirements by utilizing lightweight deep learning architectures rather than complex, resource-intensive models like BERT or RoBERTa. Open-source tools such as TensorFlow, Django, and FastText are used to reduce software licensing costs. This approach ensures efficient model deployment, making it suitable for real-time applications with limited hardware.

3.7.3 Time

The planning phase ensures that each stage—from data collection to deployment—is structured with clear timelines. Using pre-trained FastText embeddings speeds up training while Django's modular framework facilitates faster development and integration. Automation in data preprocessing and model evaluation reduces manual effort, leading to an overall reduction in development time. The system is designed to be easily maintainable and scalable, allowing for quick updates in the future.

CHAPTER 4

SYSTEM DESIGN

4.1 System Architecture:

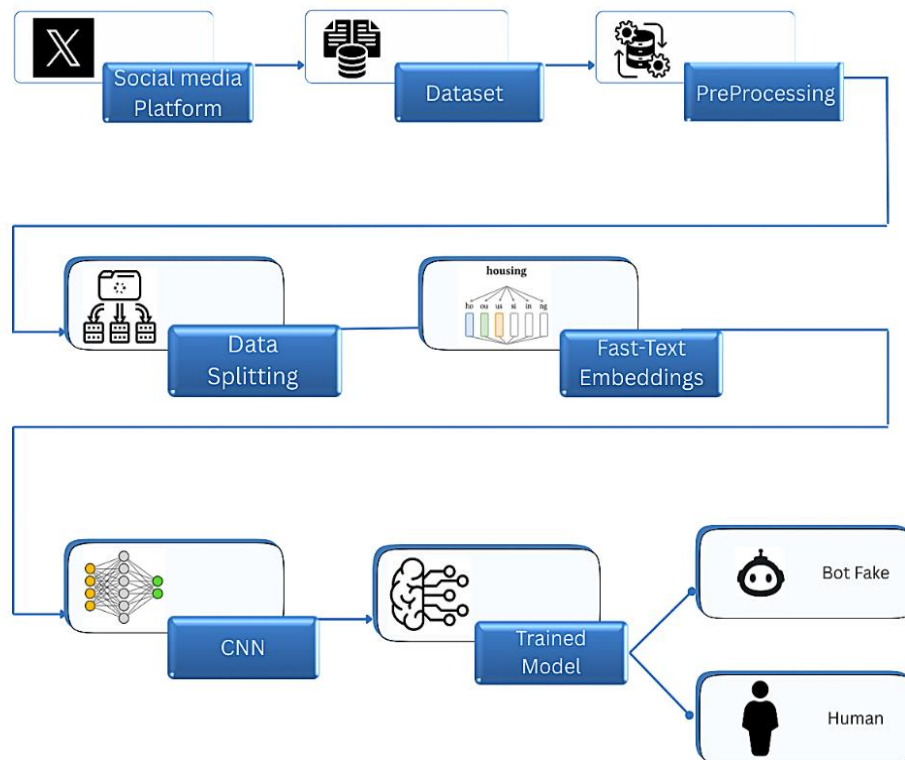


Fig. No. 4.1: System Architecture

4.1.1 Data flow diagram:

A Data Flow Diagram (DFD), also known as a bubble chart, visually represents how data moves through a system, including inputs, processing, and outputs. It helps model system components, interactions, and data transformations, making it a crucial tool for system analysis and design. By providing a structured view of data flow, DFDs enhance communication between developers, designers, and stakeholders, ensuring efficient system development.

1. Import Libraries

- The process begins by importing the necessary libraries required for data handling, preprocessing, model training, and evaluation.

2. Verification Step

- A decision is made to **verify** the dataset before processing.
- If verification **fails (No)** → The process terminates with “NO PROCESS.”
- If verification **succeeds (Yes)** → The workflow continues to data processing.

3. Importing the Dataset

- The dataset is loaded into the system for further processing.

4. Data Processing

- Raw data is cleaned, formatted, and prepared to ensure consistency and usability.

5. Feature Selection

- Important features are selected from the dataset to improve model efficiency and accuracy.

6. Splitting Data into Train & Test Sets

- The dataset is divided into training and testing sets for model evaluation.

7. Building the Model

Multiple machine learning models are considered for training, including:

- **Naïve Bayes**
- **Logistic Regression**
- **Decision Tree**
- **Random Forest**
- **Gradient Boosting**
- **Proposed CNN** (Convolutional Neural Network)
- **Extended Hybrid CNN** (possibly an optimized CNN model)

8. Training the Model

- The selected model is trained using the training dataset to learn patterns.

9. Signup & Sign-in

- Users need to create an account or log in to interact with the system.

10. User Input

- The user provides input (e.g., text, image, or other data) for prediction.

11. Outcome

- The system processes the input using the trained model and generates predictions.

12. End Process

- The process concludes after generating and displaying results.

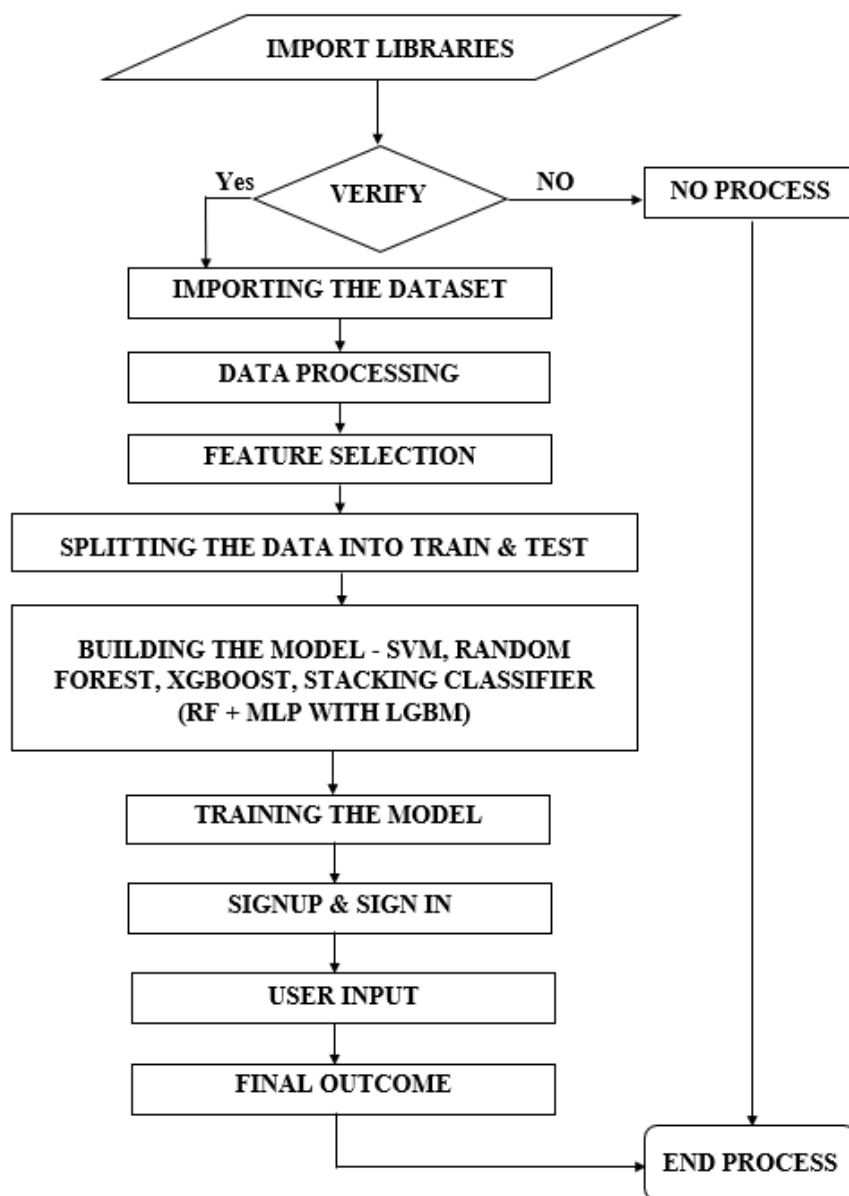


Fig. No. 4.2: Data Flow Diagram

4.1.2 UML diagram:

This UML Use Case Diagram illustrates the interaction between the user and the different functionalities of a machine learning-based system. The actor (User) performs multiple actions that are represented as use cases (ovals).

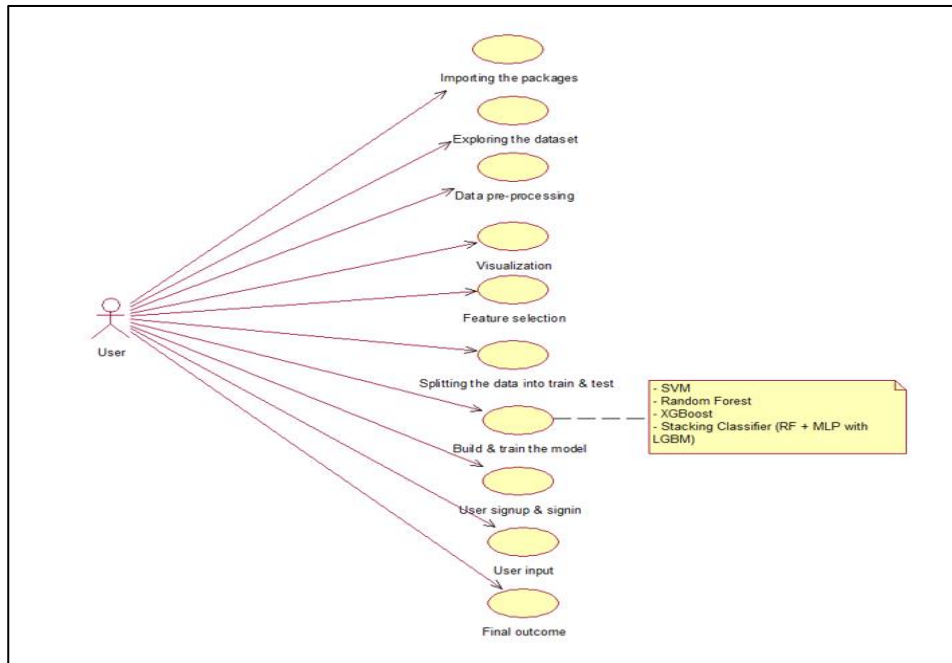


Fig. No. 4.3: UML Diagram

4.1.3 Class diagram:

The class diagram illustrates the relationships between different classes. In a class diagram, relationships between classes can be represented using either an "is-a" (inheritance) or "has-a" (composition/association) connection. They achieve their functionality through **methods**, which define their behaviour.

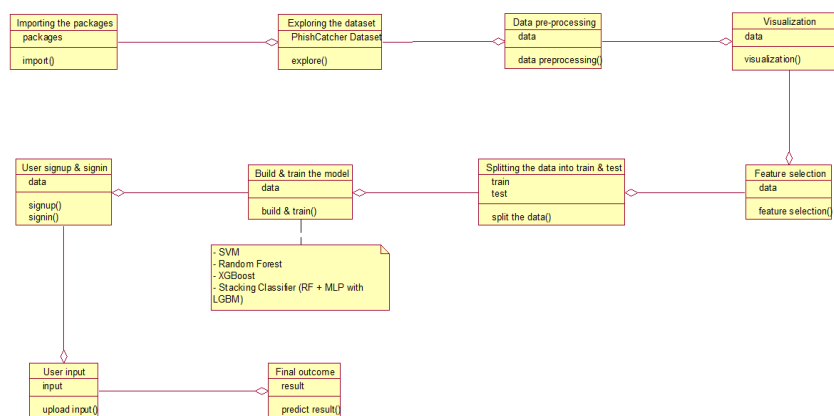


Fig. No. 4.4: Class Diagram

4.1.4 Activity diagram:

The **activity diagram** represents the process flows within a system. It is like a **state diagram** as it illustrates **guard conditions, initial and final states, tasks, activities, and transitions between them**. However, unlike state diagrams, activity diagrams focus more on **workflow representation** rather than state changes. Activity diagrams help in **visualizing the sequence of operations** in a system, making them useful for **business process modeling, software development, and system behavior analysis**. They show how different elements interact, including **decisions, loops, and parallel processes**, making them a powerful tool for understanding system logic.

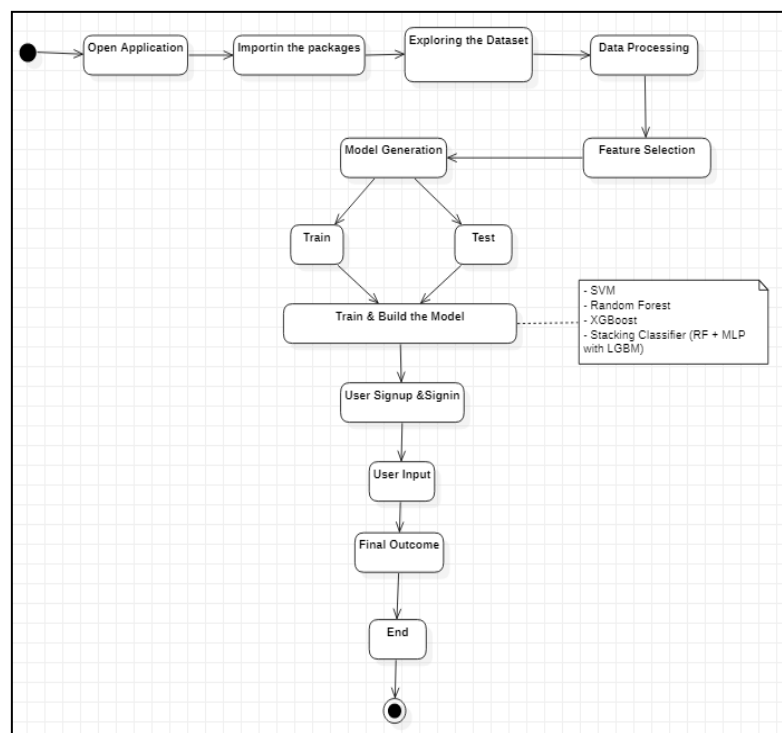


Fig. No. 4.5: Activity Diagram

4.1.5 Sequence diagram:

A **sequence diagram** visually represents the **interdependencies and interactions between various system components** over time. In a sequence diagram, different system components communicate through **"messages,"** which define the flow of information and control between them. These messages can be **synchronous or asynchronous**, depending on whether a response is required before proceeding to the next step.

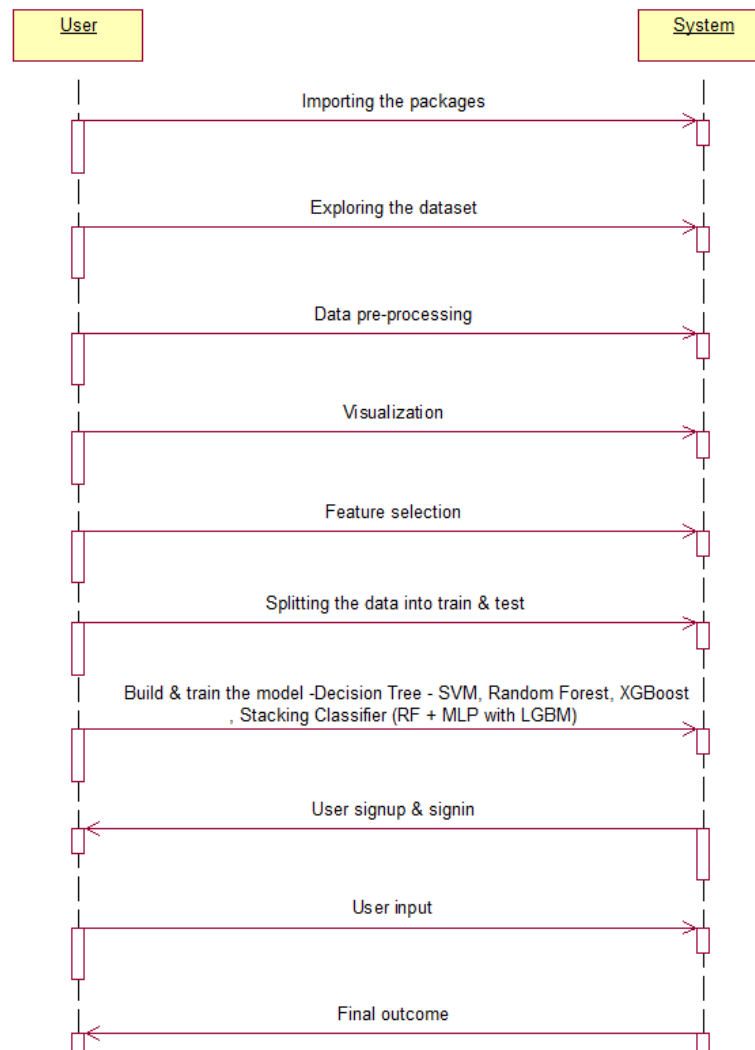


Fig. No. 4.6: Sequence Diagram

4.1.6 Collaboration diagram:

The relationships between many things may be shown in a cooperation diagram. The order of the interactions may be seen by looking at the numbered list of interactions. Every conceivable interaction between components may be shown in the cooperation diagram.

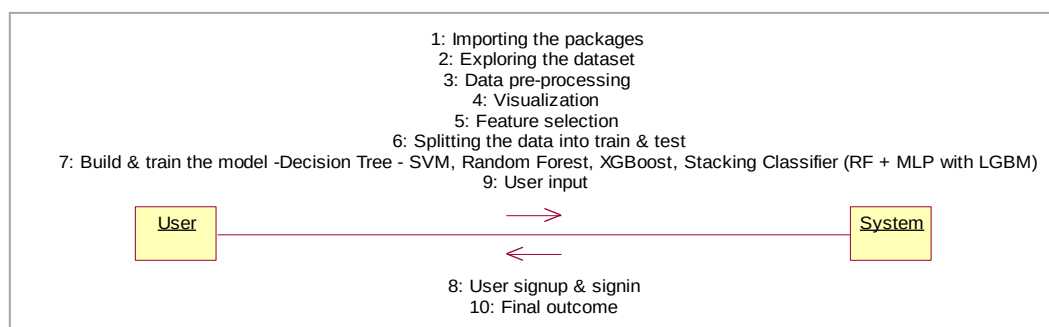


Fig. No. 4.7: Collaboration Diagram

4.1.7 Component diagram:

The system's essential components are shown in the component diagram. This schematic provides an overview of the system by showing its primary parts and their interconnections. To better understand how a built or created system works, a component diagram may be very helpful.

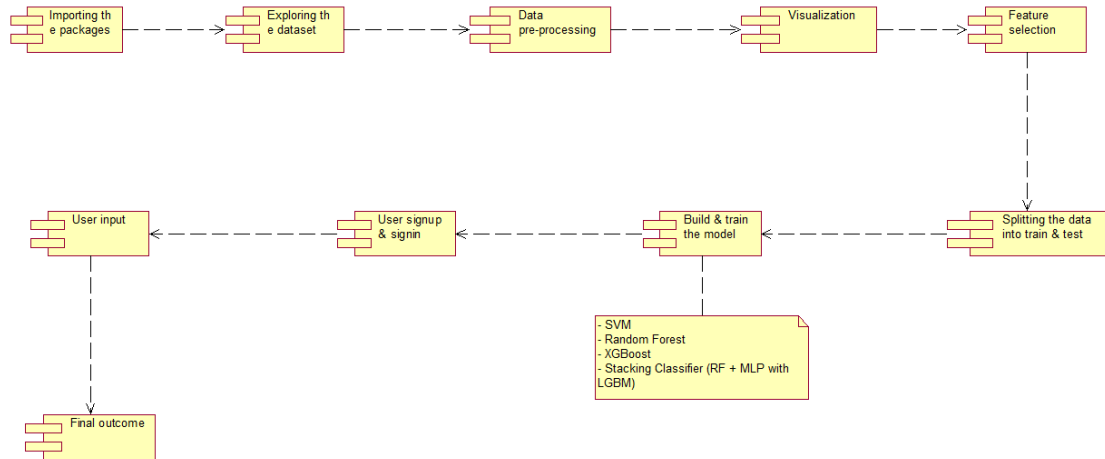


Fig. No. 4.8: Component Diagram

CHAPTER 5

IMPLEMENTATION

5.1 Implementation Setup: Python Installation

1. Download Python 3.7.6

- Go to the official Python website:
<https://www.python.org/downloads/release/python-376/>
- Download the appropriate installer for your operating system (Windows/macOS/Linux).

2. Install Python for windows:

- Run the downloaded .exe file.
- Check the box "Add Python to PATH" before proceeding.
- Click Install Now and follow the installation wizard.

3. Verify Installation:

- Use the following command in the terminal: `python --version`
- It should display `Python 3.7.6`

5.2 Implementation Setup: Environment Files

To successfully implement the project, several dependencies and configurations need to be set up. The environment setup consists of:

- **A dependency file (set1.txt)** for installing required Python packages.
- **A script (nltdownload.py)** to download necessary NLP datasets.
- **A Django management file (manage.py)** for running the web framework.

Additionally, the following set of commands is used to automate and streamline the installation and execution of the project.

5.2.1 Dependency file (set1.txt):

This file contains a list of all required Python libraries and their specific versions. The dependencies span multiple domains, including:

- **Machine Learning & Deep Learning:** tensorflow, keras, scikit-learn, xgboost, lightgbm, catboost.
- **Natural Language Processing (NLP):** nltk, textblob, gensim, pytesseract, googletrans.
- **Web Frameworks:** Django, Flask.
- **Data Processing & Visualization:** pandas, numpy, matplotlib, seaborn, scipy, plotly.

Functionality:

- Installing all required dependencies ensures that the environment is preconfigured with the necessary tools for implementation.
- The command `pip install -r set1.txt` reads the file and installs all listed libraries automatically.

Installation command:

“`pip install -r .\set1.txt`”

- **pip:** Python package manager used for installing libraries.
- **install:** Command to install packages.
- **-r:** Reads dependencies from a file.
- **.\set1.txt:** Specifies the file containing the list of required packages.

5.2.2 NLP Dataset Downloader file (nltkdownload.py):

This script ensures that the **Natural Language Toolkit (NLTK)** is fully set up by downloading all necessary datasets and corpora.

Functionality:

- Runs `nltk.download()`, which opens an interactive menu to select datasets for download.

- Ensures all necessary resources (stopwords, tokenizers, word lists, etc.) are available for text processing tasks.

What happens if this file is missing?

- The NLP models might fail due to missing linguistic datasets.
- Some functions like tokenization, stemming, and lemmatization may not work properly.
- Manual download of individual corpora would be needed, which is inefficient.

Installation command:

“python -m nltk.downloader all”

- **python:** Calls the Python interpreter.
- **-m:** Runs a module as a script.
- **nltk.downloader:** The module for downloading NLTK datasets.
- **all:** Downloads all available corpora and resources.

5.2.3 Django Project Management Script file (manage.py):

This script is a standard **Django management tool** that allows running various commands related to the Django framework.

Functionality:

- Sets the environment variable DJANGO_SETTINGS_MODULE, which links to the project's settings.
- Calls Django's command-line utility (execute_from_command_line()) to handle different tasks like running the server, applying migrations, and creating superusers.

What happens if this file is missing?

- The command py manage.py runserver will fail since it depends on this file.
- Django-related tasks such as migrations, database interactions, and server startup will be inaccessible.

Installation command:

“py .\manage.py runserver”

- **py**: Executes Python scripts.
- **.\manage.py**: Django management script for project control.
- **runserver**: Starts the local web server for Django applications.

This command initializes the Django web application, making it accessible via a browser. If the manage.py file is missing or improperly configured, the server will not start, affecting the project's deployment.

5.3 Implementation Setup: Django Environment

To set up a Django project, a series of commands are executed to create a virtual environment, install Django, and initialize the project and application. These commands collectively establish a Django development environment, ensuring smooth project creation and execution. Below is a detailed explanation of each command that should be executed in sequence to form a proper Django directory structure:

1. **python -m venv environment**

- This command creates a virtual environment named "**environment**" using Python's built-in venv module.
- A virtual environment isolates dependencies, ensuring that project-specific packages do not interfere with system-wide installations.

2. **.\environment\Scripts\activate**

- This activates the virtual environment "**environment**", allowing Python and pip to use the environment's dependencies instead of system-wide packages.
- On macOS/Linux, the command would be source environment/bin/activate.

3. **pip install Django**

- Installs Django inside the virtual environment to ensure the project uses a consistent version of the framework.

4. django-admin --version

- Checks if Django was installed correctly by displaying the installed version.

5. django-admin startproject Deep

- Creates a new Django project named "**Deep**".
- This command generates essential project files, including manage.py, a settings module, and initial configurations.

6. cd Deep

- Navigates into the newly created project directory "**Deep**", allowing further configurations and application setup.

7. django-admin startapp Deepapp

- Creates an application named "**Deepapp**" inside the project.
- In Django, an application represents a functional module within the project (e.g., user authentication, blog, etc.).

8. python manage.py runserver

- Starts the Django development server, allowing developers to access the project in a web browser at <http://127.0.0.1:8000/>.

5.4 Implementation: CNN Model

The CNN model is loaded in views.py from a .hdf5 file and used to classify tweets. While the full model definition is likely done during training, let's reconstruct the architecture typically used for text classification with CNN + TF-IDF in mind.

- Input: TF-IDF-transformed vectors
- Output: Binary classification (0 → Normal, 1 → Bot)
- Architecture: A simple 1D CNN tailored for text classification

The model/ directory serves as a storage hub for pre-trained models, feature extraction tools, and preprocessed datasets, ensuring the system runs efficiently without the need for redundant computations in the CNN model architecture.

1. **cnn_history.pkl:** This file stores the training history of the Convolutional Neural Network (CNN) model. It is created using Python's pickle module during the training phase of the model. It is used during performance analysis to visualize or evaluate how well CNN learned during training. For example, you can plot loss vs. epochs or accuracy vs. epochs using the values from this file.

Contents:

- loss: Training loss values per epoch.
- accuracy: Training accuracy values per epoch.
- val_loss: Validation loss values per epoch (if validation is used).
- val_accuracy: Validation accuracy values per epoch.

2. **cnn_weights.hdf5:** This file stores the learned weights of the CNN model after training, saved in the HDF5 format. Instead of retraining the model every time the app runs, the model loads these weights using `model.load_weights('cnn_weights.hdf5')` during inference.

Contents:

- The values of all trainable parameters (weights and biases) of the CNN layers.
- Model configuration (optional, if `model.save()` is used instead of `model.save_weights()`).

3. **tfidf.pkl:** This file stores a trained TF-IDF (Term Frequency-Inverse Document Frequency) vectorizer. During inference, the input tweets are transformed into TF-IDF feature vectors using this saved vectorizer to ensure compatibility with the trained model.

Contents:

- Vocabulary and IDF scores for each term in the corpus.
- The method to transform new input text into the same vector space.

4. **X.npy & Y.npy:** These files store the preprocessed dataset used for training the model in NumPy binary format for fast loading. Pre-saving these avoids repeating preprocessing steps like tokenization and vectorization every time.

Contents:

- X.npy: Contains the TF-IDF feature matrix (i.e., vectorized tweet data).
- Y.npy: Contains corresponding labels (e.g., 0 for 'normal', 1 for 'bot').

5.4.1 Prediction Flow in views.py

Load Model Architecture, Weights, and TF-IDF Vectorizer: The first step is to reconstruct the CNN model architecture in code. This includes layers such as:

- Input layer matching the TF-IDF input shape.
- 1D Convolutional layers with ReLU activation for feature extraction.
- MaxPooling layers for downsampling.
- Flatten and Dense layers for final classification.

Once the architecture is defined, the model loads its trained weights from the file `cnn_weights.hdf5` using the `load_weights()` method. Simultaneously, the pre-trained TF-IDF vectorizer is loaded from `tfidf.pkl` using `pickle.load()`. This vectorizer ensures consistent text-to-feature transformation as used during training.

Receive Tweet Input from User: When the user submits a tweet through the web interface, a POST request is sent to the corresponding Django view. The tweet is accessed in `views.py` using:

```
“input_text = request.POST['tweet']”
```

This raw text input is now ready for preprocessing and prediction.

Preprocess Input Using TF-IDF Vectorizer: The loaded TF-IDF vectorizer transforms the user-provided tweet from raw text into a numerical vector. This vector captures the importance of words (or tokens) in the context of the dataset used for training. The transformation looks like:

```
“transformed_input = tfidf_vectorizer.transform([input_text]).toarray()”
```

The result is a fixed-length vector, which serves as the input to the CNN.

Pass Input to CNN Model for Prediction: The TF-IDF vector is reshaped as needed and passed to the CNN model using:

```
“prediction = cnn_model.predict(transformed_input)”
```

The CNN processes this vector through multiple layers to extract relevant patterns and makes a final prediction. The output is a probability score between 0 and 1, indicating how likely the tweet is from a bot.

Interpret the Prediction Output: The final probability output from the CNN is interpreted as:

- If ≥ 0.5 → The tweet is classified as a Bot-generated tweet
- If < 0.5 → The tweet is classified as a Normal (Human) tweet

5.5 Implementation: Backend (views.py)

The views.py file serves as the central logic controller of the Django web application. It defines how the server responds to various user requests by connecting user interfaces (templates) to machine learning models, datasets, and backend computations. In this deepfake tweet detection system, views.py plays a vital role by handling multiple essential operations that contribute to the overall functionality and interactivity of the application. The file facilitates communication between the frontend and backend, ensuring that user inputs are processed accurately and that results are dynamically rendered onto the web pages. It also manages routing logic, conditional rendering of results based on form submissions, handling different types of user requests (such as POST or GET), and displaying outputs such as model performance metrics, prediction results, and graphs. Each function within the file is designed to handle a specific part of the system, ensuring modularity, scalability, and maintainability of the codebase. The detailed breakdown of the backend implementation is as follows:

1. Global Imports and Initializations

```
from django.shortcuts import render, redirect
from django.contrib.auth.models import User
from django.contrib.auth import authenticate, login
import numpy as np
import pandas as pd
import os
import pickle
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
from sklearn.naive_bayes import MultinomialNB
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier
from keras.models import Sequential, Model
from keras.layers import Conv2D, MaxPooling2D, Flatten, Dense
from keras.utils import to_categorical
from keras.models import load_model
import nltk
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer, PorterStemmer
import string
import matplotlib.pyplot as plt
import base64
from io import BytesIO
```

This is a collection of import statements typically found at the top of a Django view file, particularly in a machine learning or deep learning project integrated into a Django web application. This prepares everything needed for a **web-based ML/NLP application** where users can interact with trained models through a web interface.

2. cleanText Function

```
def cleanText(doc):
    tokens = doc.split()
    table = str.maketrans('', '', string.punctuation)
    tokens = [w.translate(table) for w in tokens]
    stop_words = set(stopwords.words('english'))
    lemmatizer = WordNetLemmatizer()
    ps = PorterStemmer()
    tokens = [ps.stem(lemmatizer.lemmatize(word.lower())) for word in tokens if word.isalpha() and word not in stop_words]
    return ' '.join(tokens)
```

The cleanText function performs several cleaning steps: it first splits the text into individual words (tokens), then removes punctuation from each word. After that, it filters out common English stopwords (like "the", "is", etc.), converts the remaining words to lowercase, applies lemmatization to reduce words to their base form, and then stemming to further reduce them to their root form. Finally, it joins the cleaned tokens back into a single string and returns the cleaned version of the original text.

3. Dataset Preprocessing and Vectorization

```
if os.path.exists('model/X.npy'):
    X = np.load('model/X.npy')
    Y = np.load('model/Y.npy')
    with open('model/tfidf.pckl', 'rb') as f:
        tfidf = pickle.load(f)
else:
    dataset = pd.read_csv("Tweepfake.csv")
    dataset.fillna("", inplace=True)
    dataset["clean_text"] = dataset["text"].apply(cleanText)
    dataset["label"] = dataset["account.type"].apply(lambda x: 1 if x == "bot" else 0)
    from sklearn.feature_extraction.text import TfidfVectorizer
    tfidf = TfidfVectorizer()
    X = tfidf.fit_transform(dataset["clean_text"]).toarray()
    Y = dataset["label"].values
    np.save('model/X.npy', X)
    np.save('model/Y.npy', Y)
    with open('model/tfidf.pckl', 'wb') as f:
        pickle.dump(tfidf, f)
```

This code checks if preprocessed data and a saved TF-IDF model already exist in the model directory. If the files X.npy, Y.npy, and tfidf.pckl are present, it loads the feature matrix X, label array Y, and the TF-IDF vectorizer using NumPy and pickle, saving time by avoiding repeated processing. If these files are not found, it loads a CSV file named Tweepfake.csv, fills missing values, cleans the text using the cleanText function, and creates binary labels where "bot" accounts are labeled as 1 and others as 0. Then, it converts the cleaned text into numerical features using TfidfVectorizer, stores the features in X, labels in Y, and saves them to disk for future use. This ensures that the text data is only processed once and can be reused efficiently in later runs.

4. Data Scaling and Splitting

```
scaler = StandardScaler()
X = scaler.fit_transform(X)
X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2, random_state=42)
```

This code standardizes the feature data X using StandardScaler, which scales the values so that they have a mean of 0 and standard deviation of 1, improving model performance. After scaling, it splits the dataset into training and testing sets using train_test_split, where 80% of the data is used for training and 20% for testing. The random_state=42 ensures consistent results across runs by setting a fixed seed for randomness.

5. calculateMetrics Function

```
def calculateMetrics(algorithm, prediction, testY):
    accuracy[algorithm] = accuracy_score(testY, prediction) * 100
    precision[algorithm] = precision_score(testY, prediction) * 100
    recall[algorithm] = recall_score(testY, prediction) * 100
    fscore[algorithm] = f1_score(testY, prediction) * 100
```

The calculateMetrics function computes performance metrics for a given machine learning algorithm. It takes the algorithm's name, its predictions, and the true labels (testY) as inputs. It calculates four key metrics: **accuracy**, **precision**, **recall**, and **F1-score**, using scikit-learn functions, multiplies them by 100 to express them as percentages, and stores the results in corresponding dictionaries (accuracy, precision, recall, fscore) using the algorithm name as the key. This helps in comparing the performance of multiple models easily.

6. Classical ML Model Training

```
# Naive Bayes
nb = MultinomialNB()
nb.fit(X_train[:1000], Y_train[:1000])
pred = nb.predict(X_test[:200])
calculateMetrics("Naive Bayes", pred, Y_test[:200])

# Logistic Regression
lr = LogisticRegression()
lr.fit(X_train[:1000], Y_train[:1000])
pred = lr.predict(X_test[:200])
calculateMetrics("Logistic Regression", pred, Y_test[:200])

# Decision Tree
dt = DecisionTreeClassifier()
dt.fit(X_train[:1000], Y_train[:1000])
pred = dt.predict(X_test[:200])
calculateMetrics("Decision Tree", pred, Y_test[:200])

# Random Forest
rf = RandomForestClassifier()
rf.fit(X_train[:1000], Y_train[:1000])
pred = rf.predict(X_test[:200])
calculateMetrics("Random Forest", pred, Y_test[:200])

# Gradient Boosting
xgb = XGBClassifier()
xgb.fit(X_train[:1000], Y_train[:1000])
pred = xgb.predict(X_test[:200])
calculateMetrics("Gradient Boosting", pred, Y_test[:200])
```

This code trains and evaluates five popular machine learning classification algorithms: Naive Bayes, Logistic Regression, Decision Tree, Random Forest, and Gradient Boosting (XGBoost). For each model, a limited portion of the dataset is used—specifically, the first 1000 samples from the training data (X_train, Y_train) are used to fit the model, and predictions are made on the first 200 samples from the test set (X_test). After making predictions, the calculateMetrics function is called to evaluate each model's performance using metrics like accuracy, precision, recall, and F1-score, which are stored in corresponding dictionaries using the model name as the key. This setup helps compare the effectiveness of different algorithms on the same data slice, providing insights into which model performs best for the given task.

7. CNN Model

```
if not os.path.exists('model/cnn_weights.hdf5'):
    model = Sequential()
    model.add(Conv2D(64, (3,3), activation='relu', input_shape=(30,10,3)))
    model.add(MaxPooling2D((2,2)))
    model.add(Conv2D(128, (3,3), activation='relu'))
    model.add(MaxPooling2D((2,2)))
    model.add(Flatten())
    model.add(Dense(256, activation='relu'))
    model.add(Dense(2, activation='softmax'))
    model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
    trainX = X_train[:1000].reshape(-1,30,10,3)
    trainY = to_categorical(Y_train[:1000])
    testX = X_test[:200].reshape(-1,30,10,3)
    testY = to_categorical(Y_test[:200])
    hist = model.fit(trainX, trainY, epochs=10, validation_data=(testX, testY))
    model.save_weights("model/cnn_weights.hdf5")
    with open("model/cnn_history.pkl", "wb") as f:
        pickle.dump(hist.history, f)
else:
    model.load_weights("model/cnn_weights.hdf5")
```

This defines and trains a CNN model if the pre-trained weights file doesn't exist. The model consists of convolutional, pooling, flatten, and dense layers, ending with a softmax output for binary classification. It reshapes the input data and one-hot encodes the labels before training. After training for 10 epochs, it saves the model weights and training history. If the weights already exist, it simply loads them to skip retraining.

8. Hybrid CNN + RF

```
layer = Model(inputs=model.input, outputs=model.get_layer(index=-2).output)
features = layer.predict(X_train[:1000].reshape(-1,30,10,3))
rf = RandomForestClassifier()
rf.fit(features, Y_train[:1000])
features_test = layer.predict(X_test[:200].reshape(-1,30,10,3))
pred = rf.predict(features_test)
calculateMetrics("Hybrid CNN+RF", pred, Y_test[:200])
```

This implements a hybrid CNN model. It first creates an intermediate model that outputs feature from the second-last layer of the trained CNN. Using this layer, it extracts deep features from the training data (X_train[:1000]) by reshaping it and passing it through CNN. The performance of this hybrid model is then evaluated using the calculateMetrics function and stored under the name "Hybrid CNN+RF".

9. Django View Functions for Web Interface

```
def LoadDataset(request):
    data = pd.read_csv("Tweepfake.csv").head(100)
    return render(request, "ViewOutput.html", {"data": data})

def FastText(request):
    return render(request, "ViewOutput.html", {"data": X})

def TrainML(request):
    labels = list(accuracy.keys())
    acc = list(accuracy.values())
    fig = plt.figure()
    plt.bar(labels, acc)
    plt.ylabel("Accuracy")
    plt.xticks(rotation=45)
    plt.tight_layout()
    buffer = BytesIO()
    fig.savefig(buffer, format='png')
    buffer.seek(0)
    image_png = buffer.getvalue()
    buffer.close()
    graph = base64.b64encode(image_png).decode('utf-8')
    return render(request, "ViewGraph.html", {"accuracy": accuracy,
                                              "precision": precision,
                                              "recall": recall,
                                              "fscore": fscore,
                                              "graph": graph})

def DetectFake(request):
    return render(request, "DetectFake.html")
```

```
def DetectFakeAction(request):  
    if request.method == "POST":  
        tweet = request.POST.get("tweet")  
        temp = tfidf.transform([cleanText(tweet)]).toarray().reshape(-1,30,10,3)  
        model.load_weights("model/cnn_weights.hdf5")  
        prediction = model.predict(temp)  
        label = "Normal Tweet" if np.argmax(prediction) == 0 else "Bot Fake Tweet"  
        return render(request, "DetectFake.html", {"prediction": label})
```

- The **LoadDataset function** reads the first 100 rows from the Tweepfake.csv dataset using pandas and passes this data to the ViewOutput.html template for display.
- The **FastText function** sends the pre-processed feature matrix X to the same ViewOutput.html template. This allows users to view the transformed text data.
- The **TrainML function** visualizes and displays the accuracy of various machine learning models. It generates a bar chart of model names vs. their accuracy scores using matplotlib, encodes the chart as a base64 image, and passes it to the
- ViewGraph.html template along with other performance metrics.
- The **DetectFake function** simply renders the DetectFake.html template, which likely contains a input area where users can submit tweets for fake tweet detection.
- The **DetectFakeAction function** processes the tweet entered by the user. It cleans the text, transforms it using the saved TF-IDF vectorizer, reshapes it for CNN input, loads the trained CNN model weights, makes a prediction as either a "Normal Tweet" or "Bot Fake Tweet". The result is then shown on the same detection page.

10. Authentication Views

```
def index(request):  
    return render(request, 'index.html')  
  
def UserLogin(request):  
    return render(request, 'UserLogin.html')  
  
def Register(request):  
    return render(request, 'Register.html')  
  
def UserScreenView(request):  
    return render(request, 'UserScreen.html')
```

```
def UserLoginAction(request):
    if request.method == "POST":
        email = request.POST.get('email')
        password = request.POST.get('password')
        try:
            username = User.objects.get(email=email.lower()).username
            user = authenticate(username=username, password=password)
            if user is not None:
                login(request, user)
                return render(request, "UserScreen.html")
            else:
                return render(request, 'UserLogin.html', {"error": "Invalid Credentials"})
        except:
            return render(request, 'UserLogin.html', {"error": "User not found"})

def UserRegisterAction(request):
    if request.method == "POST":
        name = request.POST.get("name")
        email = request.POST.get("email")
        password = request.POST.get("password")
        if not User.objects.filter(email=email).exists():
            User.objects.create_user(username=name, email=email, password=password)
            return render(request, "UserLogin.html")
        else:
            return render(request, "Register.html", {"error": "User already exists"})
```

- The **index function** renders the homepage by returning the index.html template when a user accesses the root URL of the application.
- The **UserLogin function** renders the UserLogin.html template, providing a login form for the user to enter their email and password.
- The **Register function** loads the Register.html template, which displays a registration form for new users to sign up by entering their name, email, and password.
- The **UserScreenView function** renders the UserScreen.html template, typically shown after successful login, acting as the user's dashboard or landing page.
- The **UserLoginAction function** handles the login form submission. It retrieves the email and password from the POST request, finds the corresponding username using the email, authenticates the user, and logs them in if the credentials are valid. If authentication fails or the user is not found, it displays an appropriate error message on the login page.
- The **UserRegisterAction function** handles user registration. It gets the name, email, and password from the form and checks whether the email already exists in

the system. If not, it creates a new user and redirects them to the login page. If the email is already registered, it shows an error message on the registration page.

5.6 Implementation: Template – View Mapping (urls.py)

The URL mapping is the process of connecting a specific Template to a view function that handles the request and returns a response. Each path in this file is associated with a particular view function, using the `path()` function. The `name` parameter allows the path to be referenced easily in templates using `{% url %}` tag. The application-level `urls.py` only handles routes for that specific app, and it must be included in the project-level `urls.py` using `include()` function. This setup allows each app to manage its own URL patterns, keeping the project well-organized and scalable.

Application level URL mapping:

```
from django.urls import path
from .views import *
from . import views

urlpatterns = [path("", views.index, name="index"),
               path("UserLogin.html", views.UserLogin, name="UserLogin"),
               path("UserLoginAction", views.UserLoginAction, name="UserLoginAction"),
               path('register/', UserRegisterAction, name='UserRegisterAction'),
               path('user-screen/', views.UserScreenView, name='UserScreen'),
               path("LoadDataset", views.LoadDataset, name="LoadDataset"),
               path("FastText", views.FastText, name="FastText"),
               path("TrainML", views.TrainML, name="TrainML"),
               path("DetectFake.html", views.DetectFake, name="DetectFake"),
               path("DetectFakeAction", views.DetectFakeAction, name="DetectFakeAction"),
               path('admin-login/', admin_login, name='admin_login'),
               path('users/', list_users, name='list_users'),
               path('users/delete/<int:user_id>', delete_user, name='delete_user'),
               path('admin-logout/', admin_logout, name='admin_logout'),
               ]
```

- `/` → `index`: Loads the homepage or landing screen of the application, typically the first screen the user sees.
- `/UserLogin/` → `UserLogin`: Displays the login form where users can input their credentials to access their account.
- `/UserLoginAction/` → `UserLoginAction`: Handles and verifies the submitted login credentials and redirects users based on authentication success or failure.
- `/Register/` → `Register`: Registration form for new users to create an account.

- `/UserRegisterAction/` → `UserRegisterAction`: Processes the submitted registration data and stores the new user details in the database.
- `/admin_login/` → `admin_login`: Presents and processes the admin login page, allowing administrators to authenticate.
- `/UserScreenView/` → `UserScreenView`: Displays the main user dashboard after successful login, serving as the primary user interface.
- `/list_users/` → `list_users`: Allows the admin to view a list of all registered users in the system.
- `/LoadDataset/` → `LoadDataset`: Loads the raw tweet dataset and displays a preview (usually first 100 rows) to the user.
- `/FastText/` → `FastText`: Displays TF-IDF or feature-extracted representation of the text dataset used in model training.
- `/TrainML/` → `TrainML`: Trains machine learning and CNN models and generates a graph comparing their performance metrics like accuracy.
- `/DetectFake/` → `DetectFake`: Renders the form where users can input a tweet to be analyzed for fakeness or authenticity.
- `/DetectFakeAction/` → `DetectFakeAction`: Processes the input tweet submitted by the user and classifies it as a real or bot-generated tweet using the CNN model.

Project level URL mapping:

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('DeepApp.urls')),
]
```

- **`path('admin/', admin.site.urls)`**: Maps the URL path `/admin/` to Django's built-in admin interface, allowing site administrators to manage models and data via the admin dashboard.

- **path('', include('DeepApp.urls')):** Routes all other URLs (i.e., root and app-level paths) to the `urls.py` file within the `DeepApp` application, delegating URL handling to the app's internal URL configuration.

5.7 Implementation: Frontend Web Interface

Django uses **templates** to generate dynamic HTML content for the front end and URLs to map the routing with a specific view function. The template files in the project are stored under the **templates/** directory inside the **app/** folder. These templates are rendered using Django's template engine when requested by users.

1. index.html – Home Page (Landing Page)

Purpose:

This template acts as the landing page of the Deepfake Tweet Detection web application. It introduces the purpose of the system with a prominent title and a welcoming interface for users. The page invites users to start interacting with the system through a single prominent call-to-action button ("Get Started..!") that directs them to the login page. Visually enhanced with a background image and styling, this page helps create a professional and engaging first impression. The simplicity of the page ensures users are guided directly to the next step without confusion.

Triggering View: `def index(request)`

Working:

- Displays the project title: "**Deepfake Detection for Identifying Machine-generated Tweets Using NLP and DL**" at the top in a stylized banner.
- A centred welcome message below the title enhances clarity of purpose.
- Provides a visually styled "Get Started..!" button which:
 1. Redirects users to the login page (UserLogin URL).
 2. Incorporates mouse-hover effects using inline JavaScript to improve UX.
- Includes a large splash image that provides visual context to the page.

Inputs:

- None. No user inputs or form elements are present.

Outputs:

- A static page with navigation to the user login section via a clickable button.

Model Interactions:

- None.

2. UserLogin.html – User Login Interface

Purpose:

This template facilitates the login functionality for registered users of the system. It presents a user-friendly form where individuals can enter their **Email ID** and **Password**. The page performs client-side validation via JavaScript to ensure both fields are filled before submission. If the login fails, feedback is displayed dynamically. Additionally, it offers navigation links for home and user management (typically for admin roles) and includes a prompt and link for new users to register. The form is styled for accessibility and user engagement with hover effects and modern UI components.

Triggering View: def UserLogin(request) , def UserLoginAction(request)

Working:

- Displays title banner as the landing page, maintaining visual consistency.
- Navigation bar at the top includes:
 1. Link to the **Home** page.
 2. Link to **Manage Users** (admin functionality).
- Shows a centred login form with the following fields:
 1. **Email ID** (input type: email)
 2. **Password** (input type: password)
- Client-side validation using JavaScript ensures:
 1. Both fields are non-empty before allowing form submission.

2. Alerts guide the user in case of missing inputs.

- The form submits to the view mapped by `UserLoginAction`, which handles backend authentication.
- Displays server-generated login feedback (error/success) dynamically using `{{ data|safe }}`.
- Provides a link labeled "**New user? Register here**", which redirects to the user registration view (`UserRegisterAction`).

Inputs:

- **Email ID**
- **Password**

Outputs:

- Login form submission.
- Conditional display of login status messages.
- Redirection to user dashboard on successful login (handled in backend).

Model Interactions:

- Indirect interaction through the `UserLoginAction` view, which likely queries the user model to validate credentials.

3. `UserRegister.html` – User Registration Interface

Purpose:

The `UserRegister.html` template provides a user-friendly interface for new users to register on the platform. It collects user information such as full name, email ID, and password to create a new account. This registration process is critical to enabling access to the system's personalized and restricted services. Additionally, it validates form completeness before submission and helps manage user onboarding securely and efficiently.

Triggering View: `def Register(request)`, `def UserRegisterAction(request)`

Working:

- The page is rendered when the user clicks on the "Register" link or button, triggering the `UserRegister` view.

- Displays a structured registration form with the following fields:
 1. **Name** (text input)
 2. **Email** (text input)
 3. **Password** (password input)
- Includes a **CSRF token** to protect against cross-site request forgery.
- Uses inline JavaScript validation (if present) or browser defaults to ensure the user fills in all fields before submission.
- On clicking **Submit**, the form sends a **POST** request to the URL {% url 'UserRegisterAction' %}, where the actual registration logic is handled.
- The backend processes the request:
 1. Verifies that the email is unique.
 2. Hashes the password securely before storing.
 3. Saves user data to the database.
- Any messages from the server (like registration success or failure) are passed through the data context variable and rendered dynamically using {{ data|safe }}.
- Upon successful registration, the user may be redirected to the login page.

Inputs:

- **Name** (text input)
- **Email** (email input with validation)
- **Password** (password input)

Outputs:

- Displays success or error messages after form submission using the data context variable.
- Provides a link to the user login page for existing users.

Model Interactions:

The form submission is handled in the UserRegisterAction view, which likely interacts with a user model (either Django's default User or a custom model) to save user information. The password is expected to be hashed securely before saving. If the email already exists or there's a validation failure, the appropriate message is sent back and shown on this page.

4. admin_login.html – Admin Authentication Interface

Purpose:

The admin_login.html template serves as a secure and dedicated login interface for administrators of the system. It ensures that only authorized personnel with valid administrative credentials can access and manage sensitive backend functionalities such as user management, model monitoring, or dataset operations. This separation of admin and user access is critical for maintaining system integrity, enforcing access control, and providing a secure workflow environment.

Triggering View: def admin_login(request)

Working:

- When an admin visits the admin login page, this template is loaded and displays a **login form** with:
 1. **Admin Name** input field
 2. **Password** input field
- A **CSRF token** is embedded in the form to protect against CSRF attacks.
- **On form submission:** A **POST request** is sent to the same view, which contains the logic to validate admin credentials.
- The backend compares the entered credentials with stored admin data (e.g., from the database or hardcoded credentials).
- **If the credentials are valid:** The admin is authenticated and redirected to the admin dashboard or another secured area (e.g., users_list.html).
- **If authentication fails:** An error message (like “Invalid login credentials”) is passed to the template via the error context variable and displayed dynamically above the form using {{ error }}.
- Additionally, the template includes a **navigation button** that links back to the standard **User Login** page via {% url 'UserLogin' %}.

Inputs:

- **Admin Name** (text input)
- **Password** (password input)

Outputs:

- If authentication fails, an error message is shown using `{{ error }}`.
- If successful, the view redirects the admin to the admin dashboard or homepage.

Model Interactions:

This view likely checks credentials against a hardcoded admin account or a separate admin database table. It does not appear to rely on Django's default AdminSite. Upon successful login, a session might be created for the admin to access restricted views.

5. users_list.html – Admin Interface for User Management**Purpose:**

The `users_list.html` template is a core part of the admin dashboard, enabling administrators to view and manage all registered users within the system. It provides a centralized interface for monitoring user activity, maintaining user records, and taking necessary actions, such as deleting inactive, suspicious, or unauthorized accounts. This management capability is essential for ensuring the overall health, security, and user quality of the application.

Triggering View: `def list_users(request)`

Working:

- When the admin accesses the page:
 - The associated Django view queries the database to **fetch all registered user records**. These user records (e.g., usernames, emails, and IDs) are passed to the template using a context variable like `users`.
- The template then dynamically:
 - **Displays all users** in a clean and styled **HTML table**. Each row represents a registered user and includes:
 1. **Username**
 2. **Email Address**
 3. A **“Delete” button** or link for account removal.

- The "Delete" action:
 - Is tied to a Django URL pattern named `delete_user`.
 - Sends a **GET or POST request** (depending on implementation) to this view with the user's unique ID.
 - The backend view handles user deletion from the database.
- **After deletion:** The page may be refreshed or redirected to show the updated user list, possibly with a success or failure message.
- Optionally, this interface may also be extended in the future to include:
 - Role management
 - Audit logs for admin actions

Inputs:

- None from the frontend directly, except for clicks on "Delete" which pass user IDs to the deletion view.

Outputs:

- Renders a table of all users.
- Allows deletion of a user via a GET request to the `delete_user` URL with their ID.
- Provides a logout option for the admin.

Model Interactions:

The triggering view retrieves all user records, likely from a custom User model using a query like `User.objects.all()`. When the admin clicks "Delete", the `delete_user` view performs a database operation to delete the selected user by ID.

6. UserScreen.html – User Screen Interface

Purpose:

The UserScreen.html template serves as the **main dashboard or homepage** for a user after successful login or registration. It acts as the central navigation hub, allowing users to explore and interact with the key functionalities of the deepfake tweet

detection system. This includes access to machine learning model performance results, the real-time tweet classification interface, and a logout option. It is designed to offer a seamless, intuitive user experience while reflecting the overall theme and goals of the application. By presenting users with essential actions in one place, this interface streamlines workflow and provides an entry point into both model interpretation and deepfake detection utilities.

Triggering View: `def UserScreenView(request), def UserLoginAction(request)`

Working:

- **Upon loading the UserScreen.html page:** The template displays a prominent project banner indicating the system's title: *"Deepfake Detection for Identifying Machine-generated Tweets Using NLP and DL"*.
- The top **navigation menu bar** includes clickable options that redirect to:
 1. **Model Metrics** (TrainML URL): To view comparative results of machine learning and deep learning algorithms used in the project.
 2. **Predict Deep Fake** (DetectFake URL): To test tweets for classification as normal or bot-generated using trained models.
 3. **Logout** (index URL): Redirects to the public home/login page, ending the user session.
- The body of the template includes:
 1. A **central image banner** (optional/static) for visual engagement and branding.
 2. A placeholder for backend messages using the context variable `{{ data|safe }}` to display:
 - Welcome messages
 - Notifications
 - Success or error feedback from previous operations
- Styling is applied using an external CSS file loaded via `{% static 'style.css' %}` to maintain consistency across templates.

Inputs:

- User navigation through links.
- data context variable for dynamic message display from backend.

Outputs:

- A user-friendly menu for accessing deepfake detection features.
- Option to view model metrics or initiate fake tweet prediction.

Model Interactions:

This template doesn't directly interact with models. However, the links it provides lead to views that do, e.g., TrainML would access model performance results stored from training, and DetectFake would involve a prediction pipeline that uses trained ML/DL models on input tweets.

7. ViewOutput.html – Output Display Page**Purpose**

The ViewOutput.html template is used to **present tabular or textual output** to the user after the backend has performed a specific data operation. It is typically used to display **loaded datasets**, **pre-processed samples**, or **intermediate outputs** from machine learning or NLP pipelines. This output visualization helps users or admins validate the data before moving to the next stages such as embedding, model training, or prediction. The purpose of this template extends beyond simple display — it enables **data transparency** and assists in **debugging, validation, and educational insight** into how the system interprets and processes tweets.

Triggering View: def LoadDataset(request), def FastText(request)

Working

- **Visual Setup:**
 1. Static CSS (style.css) is loaded via {% load static %}
 2. The page uses a centered layout with a structured header and menu bar styled in line with the application's theme.

- **Navigation Menu:**

1. The top menu includes links to:
 - **Load Dataset Sample** (LoadDataset)
 - **FastText Embeddings** (FastText)
 - **Algorithm Performance** (TrainML)
 - **Predict Deep Fake** (DetectFake)
 - **Home Page** (UserScreen)
 - **Logout** (index)
2. These links allow users to easily switch between core functionalities without re-authentication.

- **Data Display Section:**

1. At the center of the page, the template displays the content of the `{{data|safe}}` context variable.
2. This data is expected to be HTML-safe (rendered without escaping), and usually represents:
 - A formatted table (e.g., top few rows of the dataset)
 - Textual descriptions or summaries of the data

- **Additional Styling & Layout:**

1. Uses `<center>` tags and line breaks for visual spacing.
2. Footer padding ensures the output is visually centered regardless of content size.

Inputs

- **Data:** A pre-rendered HTML string passed from the view containing tabular or formatted output for display.

Outputs

- Displays data in the body section.
- No form or interactive input is present; it's a read-only output screen.

Model Interactions

- No direct interaction, model output might have been passed through views earlier and formatted before rendering.

8. ViewGraph.html – Model Performance Visualization Page

Purpose

The ViewGraph.html template is designed to visually present model performance comparisons using graphs or charts. It helps users analyze how different machine learning or deep learning models perform based on evaluation metrics like accuracy, precision, recall, F1-score, or training time. This graphical interface improves interpretability and decision-making by converting numerical results into an intuitive visual format. It plays a crucial role in assessing which model is best suited for detecting deepfake tweets, based on comparative analysis.

Triggering View: def TrainML(request)

Working

- **Static Content Setup:**
 1. Uses {% load static %} to bring in the external stylesheet style.css for consistent UI styling. Defines HTML metadata and layout structure with a centered header that displays the project title.
- **Navigation Menu:**
 1. Includes links to other core functionalities:
 - **Loaded Dataset Sample** (LoadDataset)
 - **Fast Text Embeddings** (FastText)
 - **Algorithm Performance** (TrainML)
 - **Predict Deep Fake** (DetectFake)
 - **Logout** (index)
 2. Allows seamless navigation for users to analyze outputs from different perspectives.

- **Graph Rendering Logic:**

1. Displays any additional message, label, or heading passed from the view using `{{ data|safe }}` (e.g., model names or metric descriptions).
2. Uses: `` to **embed the base64-encoded PNG graph** directly into the HTML without requiring file downloads or static hosting.
3. This mechanism ensures **dynamic graph rendering** even for real-time evaluations.

- **Visual Alignment:**

1. Elements like `<center>` and `<hr>` are used to keep content centered and cleanly separated.
2. Padding and spacing ensure the graph appears clearly without overlap or clutter.

Inputs

- `data`: Additional descriptive info or statistics to display alongside the graph.
- `img`: A base64 string of the graph image generated dynamically from matplotlib or another plotting library in the backend.

Outputs

- Visual chart/graph showing comparative performance of models. Optional supporting text/data below the graph.

Model Interactions

- Graph is likely based on metrics like accuracy, precision, recall, or F1-score gathered after training the ML/DL models plotted as a bar graph.

9. DetectFake.html – Tweet Testing Interface

Purpose

The DetectFake.html template serves as the interactive input interface where users can enter a tweet and submit it for deepfake detection. The goal is to determine whether the entered tweet is genuine (human-written) or machine-generated (deepfake)

or bot-written) using backend ML/CNN classification models. This interface is a crucial component of the application as it facilitates real-time testing of the system's prediction capability. It allows users to assess how effectively the system can flag suspicious or fake tweet content—an essential use case in misinformation detection.

Triggering View: `def DetectFake(request), def DetectFakeAction(request)`

Working

- **Static Styling and Script Setup:**

1. Uses `{% load static %}` to incorporate consistent CSS styling from `style.css`.
2. Adds a JavaScript `validate()` function to ensure the tweet text field isn't empty before submission, providing basic client-side form validation.

- **Form Rendering:**

1. Displays a centered, stylized input form containing:
 - A **textarea** for users to input the tweet text.
 - A **Submit button** that posts the form data to the backend.
2. The form uses: `<form action="{% url 'PredictAction' %}" method="POST" onsubmit="return validate ()">` which includes CSRF protection using `{% csrf_token %}` for security.

- **Prediction Display:**

1. After form submission, the backend processes the tweet and returns a prediction (e.g., "Fake Tweet Detected" or "Tweet is Genuine").
2. This result is rendered in the template via: `{{ data|safe }}` which allows the display of styled or formatted output.

- **Navigation Menu:**

1. The page includes quick links to:
 - **View Metrics** (TrainML)
 - **Loaded Dataset** (LoadDataset)
 - **Embeddings** (FastText)

- **Graph Comparison** (ViewGraph)
- **Back to User Dashboard** (UserScreen)
- **Logout** (index)

Inputs

- t1: The tweet text entered by the user into the form.
- CSRF token: For form security.

Outputs

- The result displayed as HTML-safe content ({{ data|safe }}) after processing.

Model Interactions

- The tweet input is passed to a trained model in the backend for classification.
- The result (e.g., "Bot Tweet" or "Normal Tweet") is returned and rendered in this template.

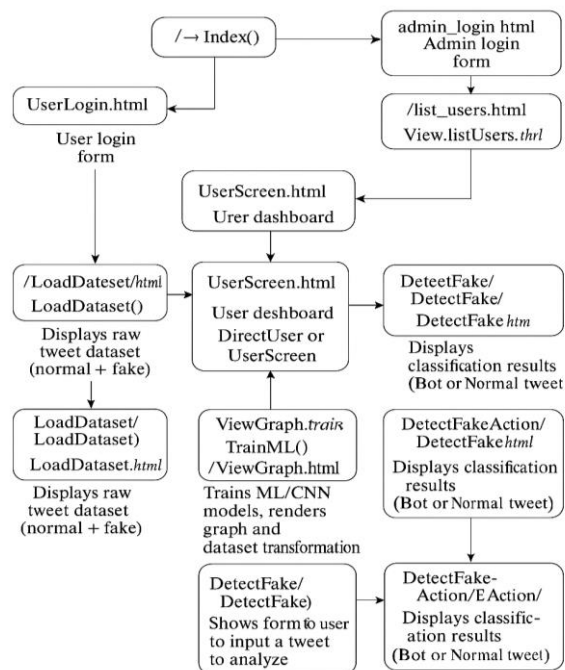
5.7.1 Summary: Template Interactions with Views (Functions)

Template Name	Triggering View Function(s)	HTTP Method(s)	Description
index.html	index	GET	Displays the landing page of the application.
UserLogin.html	UserLogin, UserLoginAction	GET, POST	Handles user login form display and submission.
UserRegister.html	Register, UserRegisterAction	GET, POST	Handles user registration form display and submission.
admin_login.html	admin_login	GET, POST	Displays and processes admin login.
users_list.html	list_users	GET	Displays a list of registered users to the admin.

UserScreen.html	UserScreenView, UserLoginAction	GET	User dashboard screen shown after successful login.
ViewOutput.html	LoadDataset, FastText	GET	Displays raw dataset or feature vector outputs.
ViewGraph.html	TrainML	GET	Displays model training results and performance graphs.
DetectFake.html	DetectFake, DetectFakeAction	GET, POST	Form to enter a tweet and displays whether it is fake or genuine after ML/CNN.

Table 5.1: Template –View Mapping

The Django application follows a Model-View-Template (MVT) pattern. Each view function controls logic and passes data to a specific HTML template, and they serve as UI interfaces for users and admin. The visual representation of the overall workflow of the mapped Django architecture, including URLs, views, and templates, is as follows:

**Fig No. 5.1: Workflow mapping of URLs, views, Templates**

5.8 Deepfake Project Directory Structure

```

Deep/
|— Dataset/
|   |— Tweepfake.csv
|
|— Deep/
|   |— __init__.py
|   |— settings.py
|   |— wsgi.py
|   |— asgi.py
|   |— urls.py
|
|— DeepApp/
|   |— __init__.py
|   |— admin.py
|   |— apps.py
|   |— models.py
|   |— tests.py
|   |— urls.py
|   |— views.py
|   |— templates/
|       |— admin_login.html
|       |— DetectFake.html
|       |— index.html
|       |— UserLogin.html
|       |— UserRegister.html
|       |— user_list.html
|       |— UserScreen.html
|       |— ViewGraph.html
|       |— ViewOutput.html
|   |— static/
|       |— style.css
|       |— templatemo_style.css
|       |— images/
|
|— model
|   |— cnn_history.pckl
|   |— cnn_weights.hdf5
|   |— tfidf.pckl
|   |— X.npy
|   |— Y.npy
|
|— requirements.txt
|— nltkdownload.py
|— set1.txt
|— download_nltk.bat
|— run.bat
|— manage.py

```

Fig. No. 5.2: Project Directory Structure

CHAPTER 6

TESTING

The testing approach for this deepfake text detection system ensures that all components function as expected, including data preprocessing, model training, classification, and web application functionalities. The testing strategy includes unit testing, integration testing, performance testing, and user acceptance testing to validate both machine learning and deep learning models, as well as the Django-based web interface.

6.1 Features to be Tested

1. **Dataset Handling** - Ensure datasets are correctly loaded, preprocessed, and stored in .npy format.
2. **Text Preprocessing** - Verify stop-word removal, tokenization, lemmatization, and vectorization (TF-IDF transformation).
3. **Machine Learning Models** - Validate the training and accuracy of classifiers such as Naïve Bayes, Logistic Regression, Decision Trees, Random Forest, and Gradient Boosting.
4. **Deep Learning Model (CNN)** - Confirm the model architecture, weight loading, feature extraction, and inference.
5. **Hybrid Model (CNN + Random Forest)** - Ensure proper feature extraction from CNN and classification with Random Forest.
6. **Web Application Views** - Test API endpoints and web interfaces for dataset loading, text detection, and authentication.
7. **User Authentication** - Validate login functionality and session management

6.2 Testing tools and Environment

Tools Used:

- **Unit Testing:** unittest (Python's built-in module)
- **Model Evaluation:** sklearn.metrics (accuracy, precision, recall, F1-score)

- **Performance Testing:** JMeter (for load testing web requests)
- **Error Logging:** pytest (for test execution and debugging)

Testing Environment:

- **Operating System:** Windows/Linux (Ubuntu 20.04 recommended)
- **Python Version:** 3.8+
- **Django Version:** 3.x or higher
- **ML/DL Frameworks:** TensorFlow/Keras, scikit-learn
- **Database:** SQLite (default Django DB) or PostgreSQL
- **Browser for Web Testing:** Chrome, Firefox

6.3 Testing Procedure

1. Unit Testing:

- Execute unit tests on text preprocessing, dataset handling, and ML/DL models.
- Validate tokenization, lemmatization, and vectorization results.

2. Integration Testing:

- Test interactions between TF-IDF vectorizer, ML models, and CNN.
- Ensure seamless data flow from dataset loading to classification.
- Validate integration of the hybrid model (CNN + Random Forest).

3. Functional Testing:

- Test web views for dataset display, text detection, and login.
- Submit sample texts via the web interface and check response correctness.

4. Performance Testing:

- Measure response time for model inference.
- Perform load testing on the web interface.
- Evaluate time taken for model training with varying dataset sizes.

5. User Acceptance Testing (UAT):

- Conduct testing with sample users.
- Gather feedback on usability, accuracy, and performance.
- Ensure that non-technical users can easily interact with the interface.

6.4 Inputs and Expected Outputs

Test case	Input	Expected Output
Dataset Loading	Tweepfake.csv	Properly loaded & processed dataset (stored in. npy format)
Text Preprocessing	Raw tweet text	Cleaned, tokenized, and lemmatized text
ML Model Training	Pre-processed dataset	Trained models with acceptable accuracy
CNN Model Training	Pre-processed dataset	CNN model with trained weights stored
Hybrid Model Execution	CNN feature-extracted dataset	Successful classification with RF model
Fake Text Detection	Input text via web form	Response: Fake or Real
User Login	Username/password	Access granted for valid users

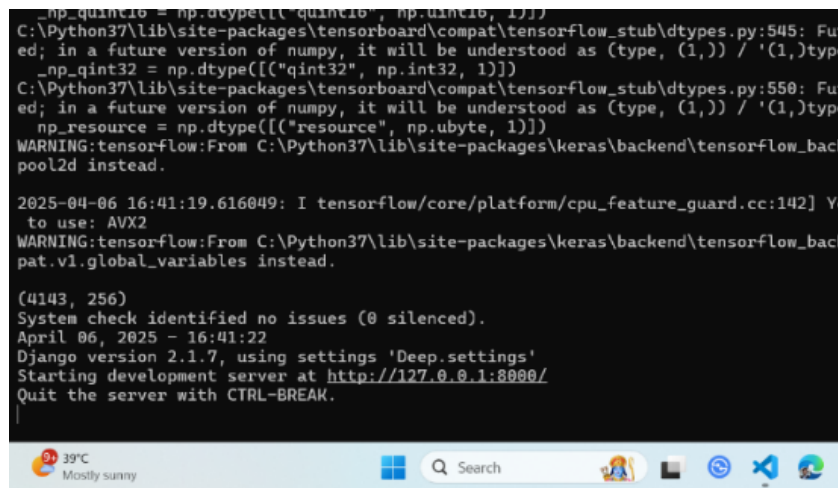
Table 6.1: Expected Outputs for each Testcase

CHAPTER 7

RESULTS

Screen 1: Running the Server

Fig. No. 7.1 Explains how to start the application. Double-click on the run.bat file to launch the Python server. Once the server is running, Ctrl + left click on <http://127.0.0.1:8000/> to access the application.



```

_np_quint16 = np.dtype([('quint16', np.uint16, 1)])
C:\Python37\lib\site-packages\tensorboard\compat\tensorflow_stub\dtypes.py:545: FutureWarning: In a future version of numpy, it will be understood as (type, (1,)) / '(1,)type'.
_np_qint32 = np.dtype([('qint32', np.int32, 1)])
C:\Python37\lib\site-packages\tensorboard\compat\tensorflow_stub\dtypes.py:550: FutureWarning: In a future version of numpy, it will be understood as (type, (1,)) / '(1,)type'.
np_resource = np.dtype([('resource', np.ubyte, 1)])
WARNING:tensorflow:From C:\Python37\lib\site-packages\keras\backend\tensorflow_backend.py:144: The name tf.nn.conv2d is deprecated. Please use tf.nn.conv2d_v2 instead.

2025-04-06 16:41:19.616049: I tensorflow/core/platform/cpu_feature_guard.cc:142] Your CPU supports instructions that this TensorFlow binary was not compiled to use: AVX2
WARNING:tensorflow:From C:\Python37\lib\site-packages\keras\backend\tensorflow_backend.py:144: The name tf.nn.conv2d is deprecated. Please use tf.nn.conv2d_v2 instead.

(4143, 256)
System check identified no issues (0 silenced).
April 06, 2025 - 16:41:22
Django version 2.1.7, using settings 'Deep.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.

```

Fig. No. 7.1: Python Server Launch in Console

Screen 2: Welcome Page

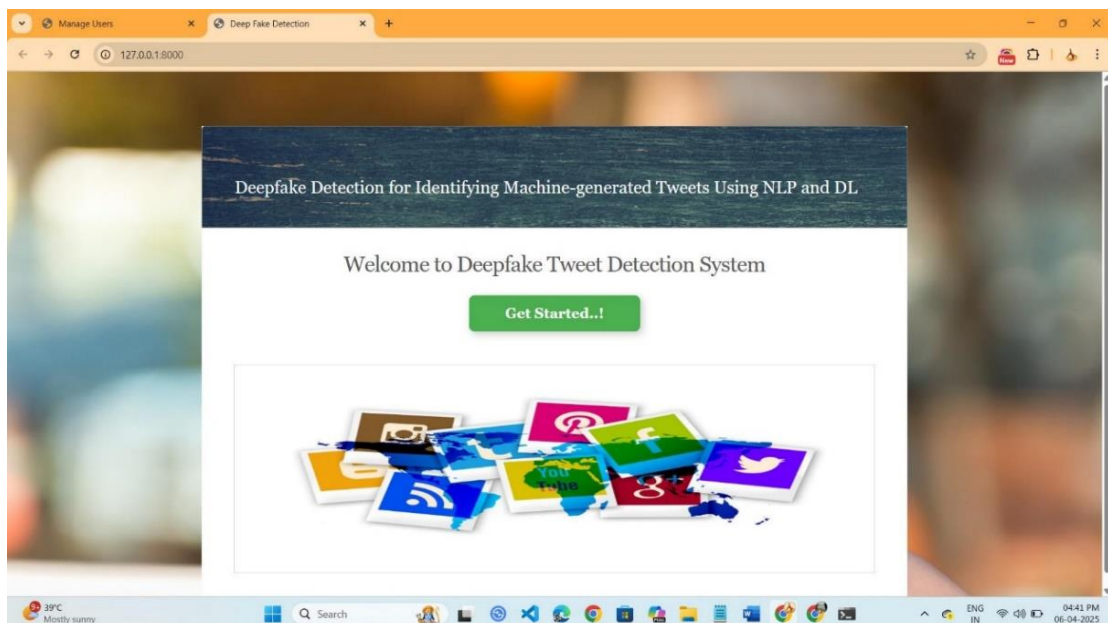


Fig. No. 7.2: Welcome page

Screen 3: Login page

Fig. No. 7.3 contains the login option for registered users, new user registration option for new users to create an ID and Admin login option to manage the users.

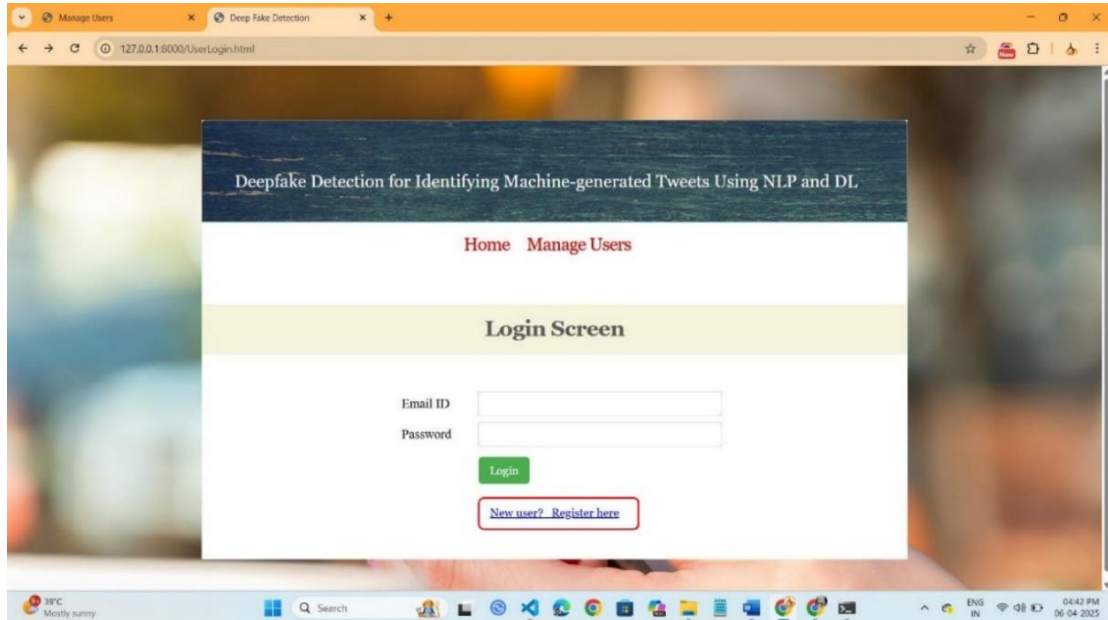


Fig. No. 7.3: Login page

Screen 4: New User Registration

Fig. No. 7.4 contains the new user registration form with name, email and password. The new users can create an ID by submitting their details.

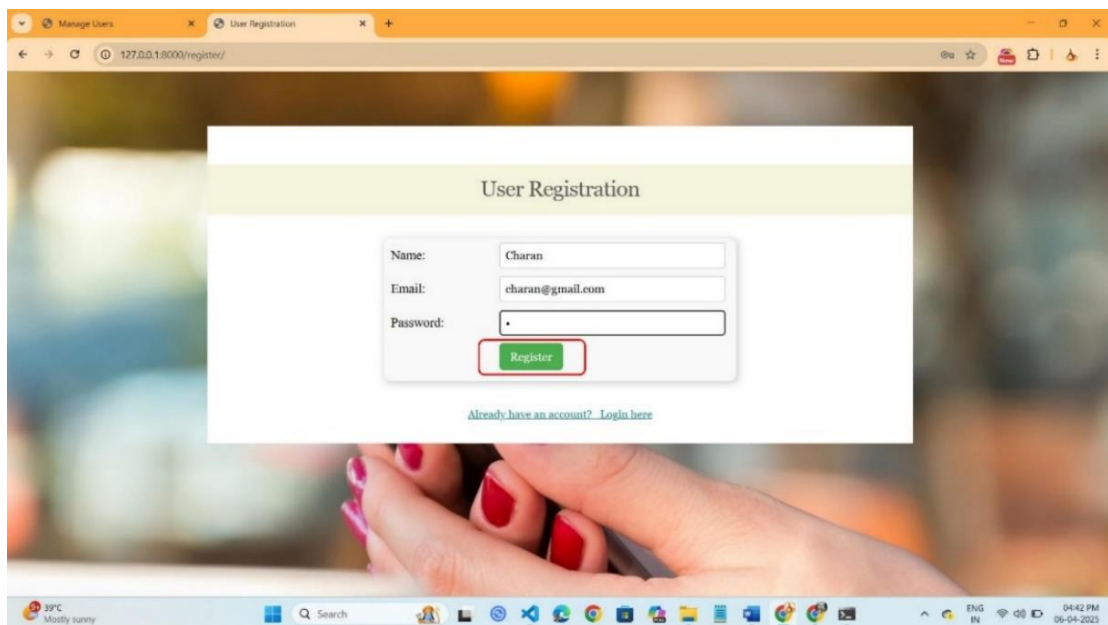


Fig. No. 7.4: User Registration page

Screen 5: Existing User Login

Fig. No. 7.5 contains the user login form where an existing user can enter their Email-Id and Password to Login and utilize the Deepfake application services.

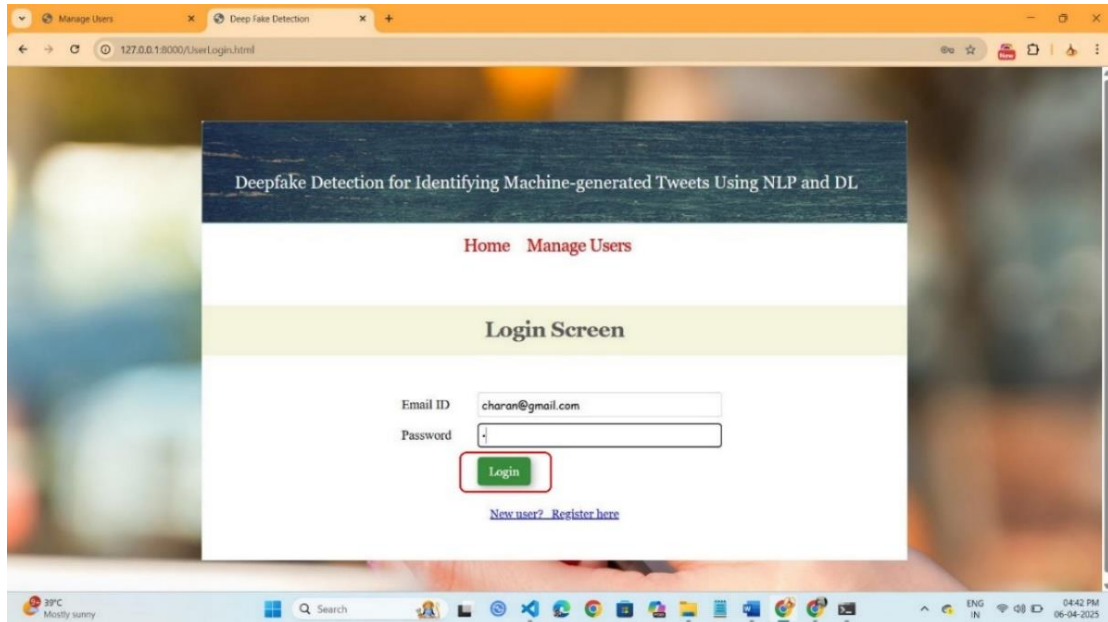


Fig. No. 7.5: Existing User Login Page

Screen 6: User Home Page

Fig. No. 7.6 Shows the home page of the application with navigation menu containing Model metrics option, Predict Deepfake option and a logout option.

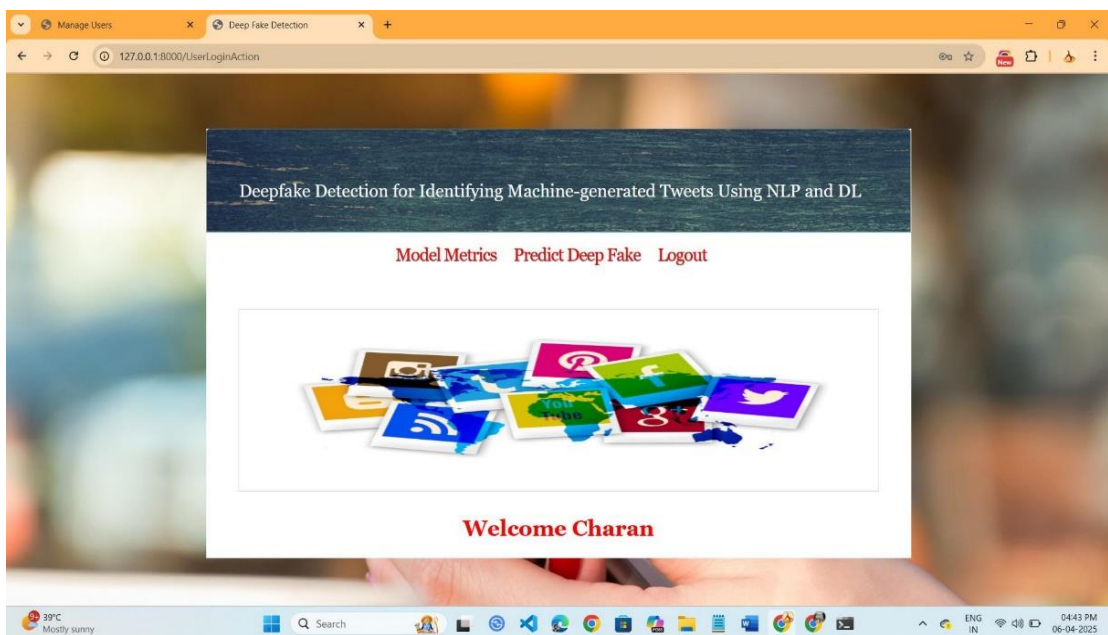


Fig. No. 7.6: User Home Page

Screen 7: Admin Login

Fig. No. 7.7 Contains the admin login form where an admin can login with the super credentials. Click on Manage users option in login page to reach this page.

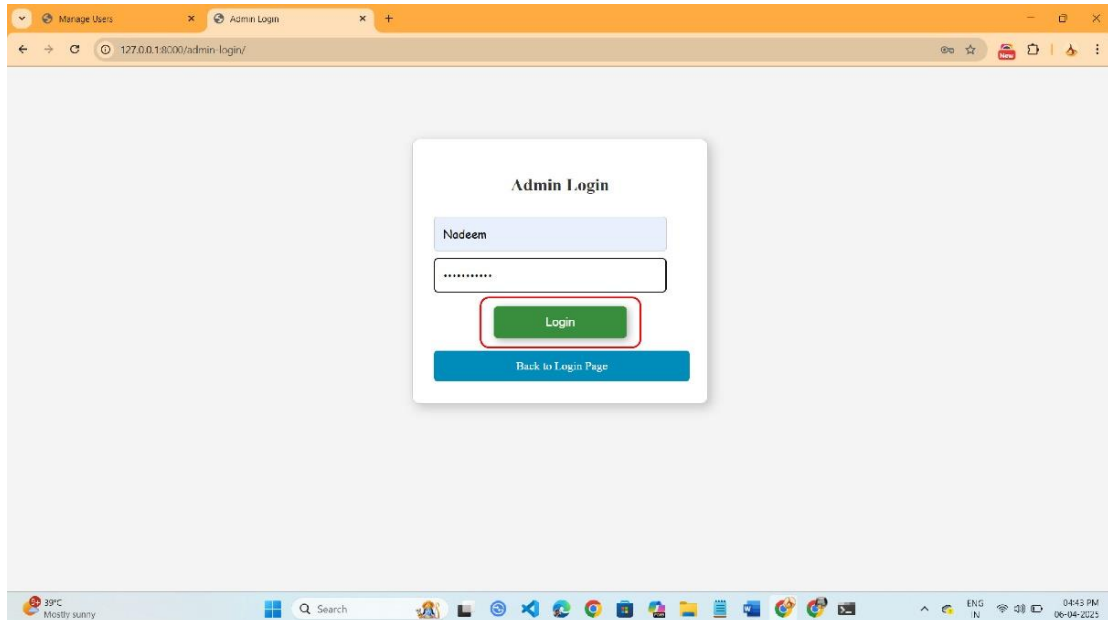


Fig. No. 7.7: Admin Login

Screen 8: User Management

Fig. No. 7.8 Contains the user table where we can see the registered users. The admin has the option to delete the ID of any registered user to remove their access.

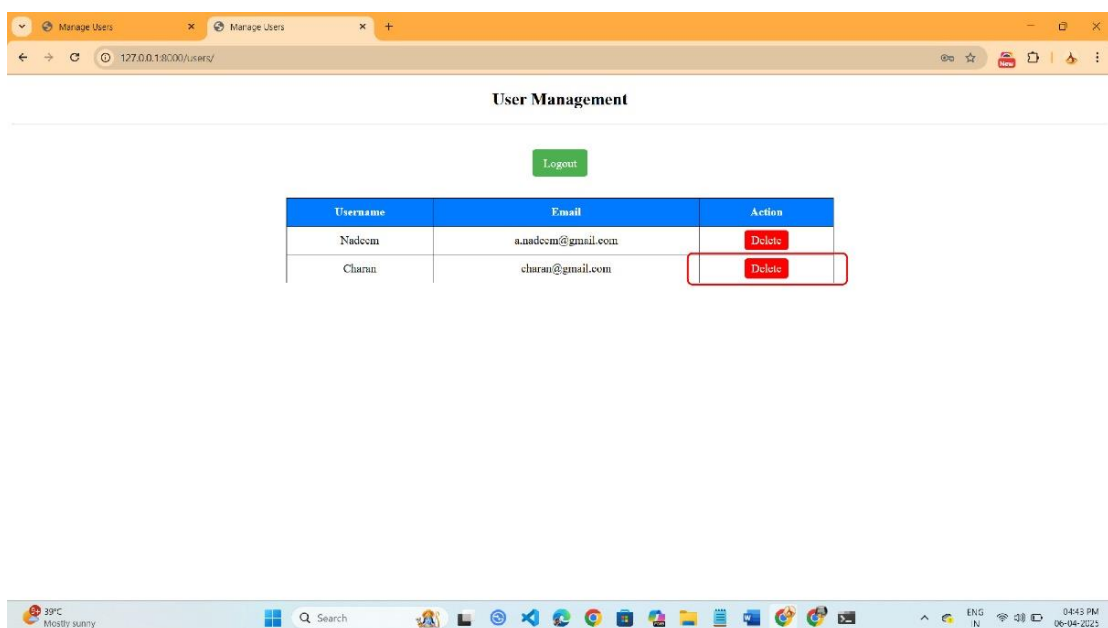


Fig. No. 7.8: Users Table

Screen 9: Home page

Fig. No. 7.9 Shows the Page after Successful login of the user. The page contains Model metrics option to explore the performance of the model, Predict option to access the prediction services of the application and a logout option.

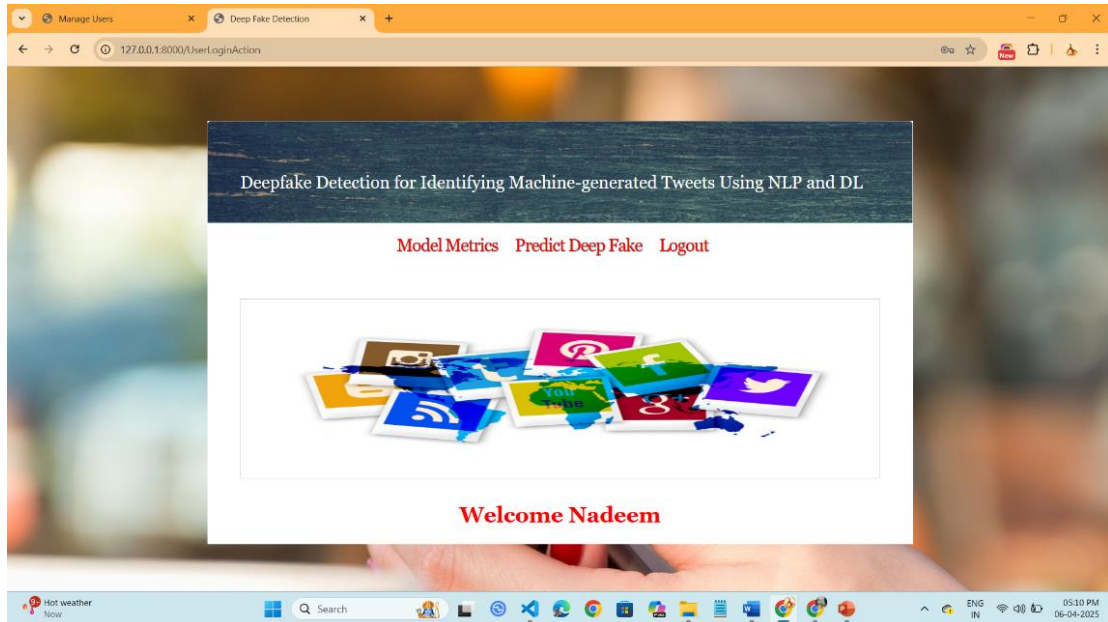


Fig. No. 7.9: Application Home Page

Screen 10: Model Metrics

Fig. No. 7.10 Shows the Comparison table, Bar graphs of the F1 scores of algorithms, We navigate to this page by clicking on Model metrics option in home page.

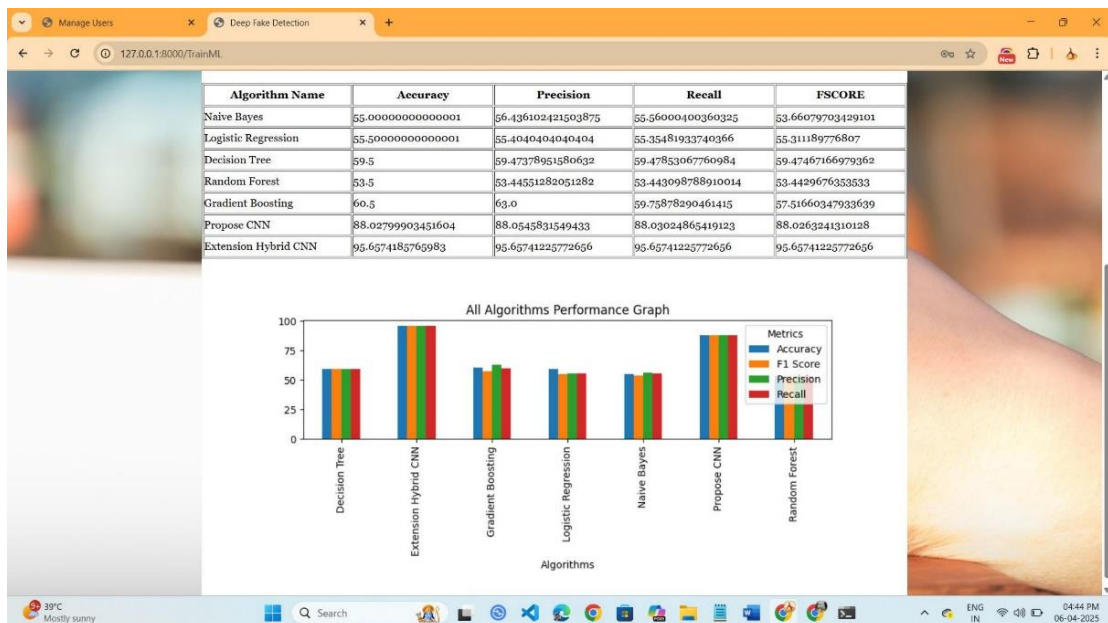


Fig. No. 7.10: Performance Graphs

Screen 11: Deepfake Prediction page

Fig. No. 7.11 Shows the Deepfake Detection service page where a user can give the tweet as input and submit it to get the output as the tweet is Normal or Botfake.

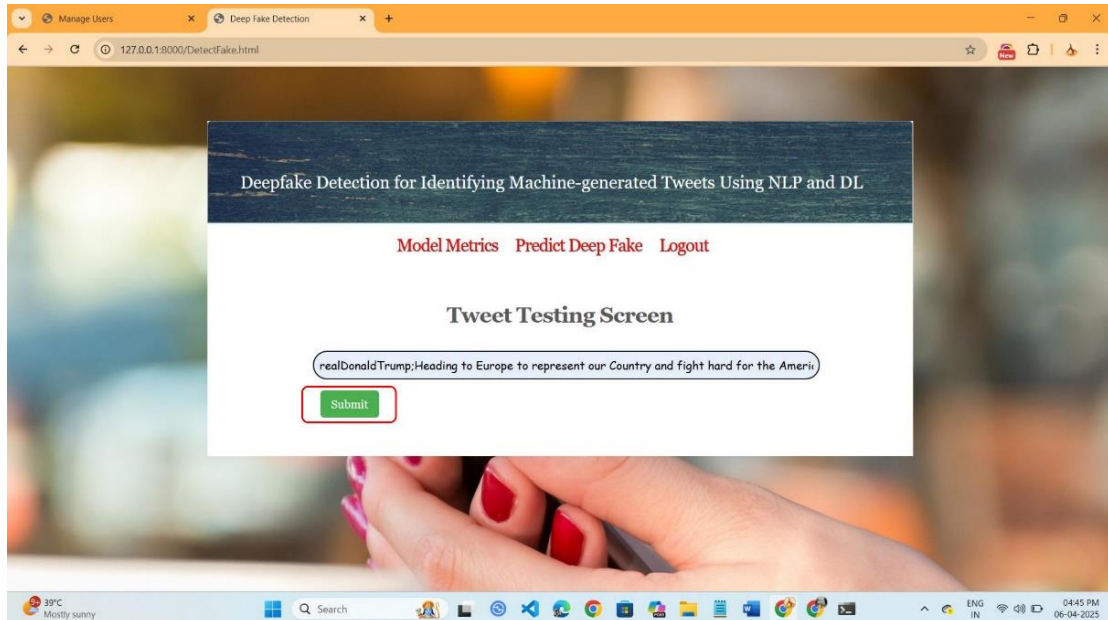


Fig. No. 7.11: Deepfake Detection page

Screen 12: Testing of Normal Tweet

Fig. No. 7.12 Shows the output as Normal as the Tweet which is given as input is said to be Human generated Tweet.

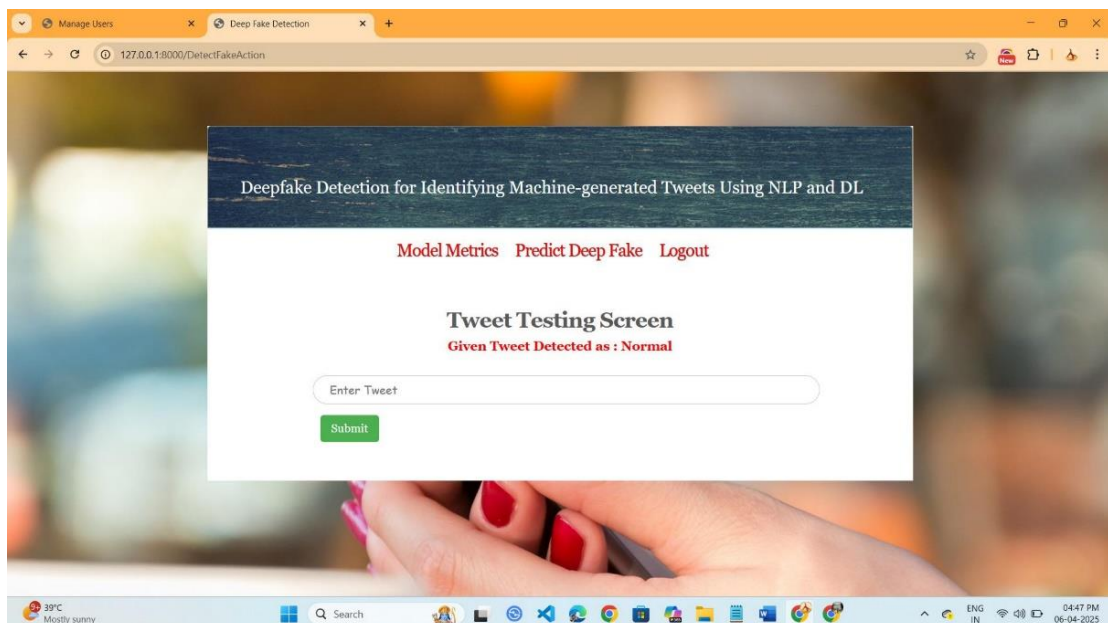


Fig. No. 7.12: Output as Normal Tweet

Screen 13: Testing of Bot Generated Tweet

Fig. No. 7.13 Shows the output as BotFake as the Tweet which is given as input is said to be a bot generated Tweet.

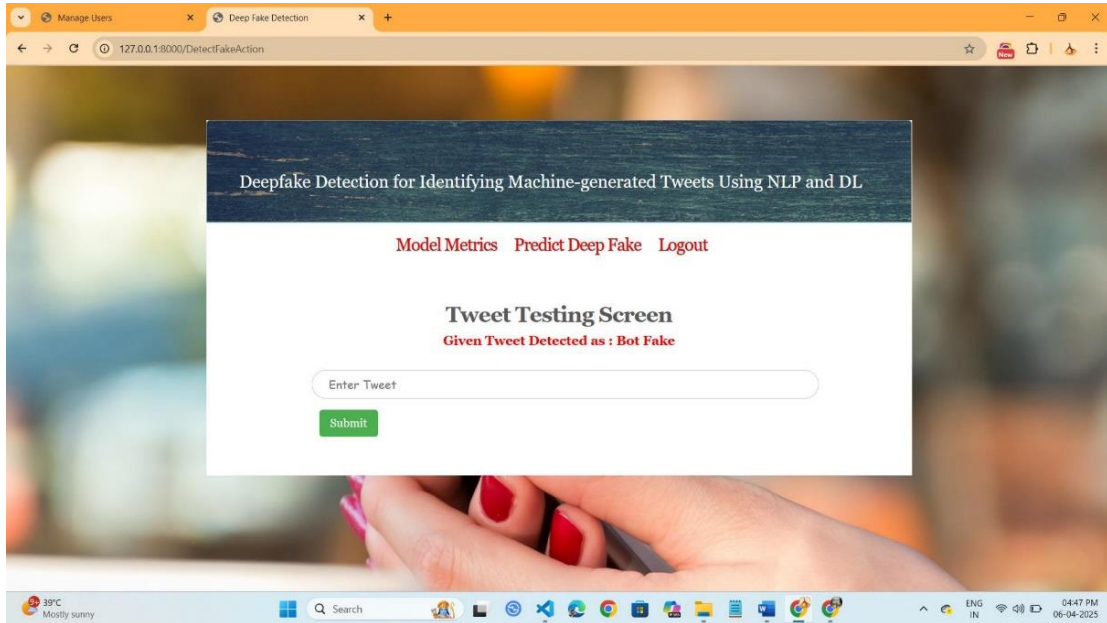


Fig. No. 7.13: Output as Bot Generated Tweet

CONCLUSION

The growing prevalence of deepfake text presents a significant challenge in today's digital landscape, where misinformation can rapidly spread across social media platforms and influence public opinion. This project successfully developed a robust deepfake text detection system by integrating machine learning, deep learning, and feature engineering techniques. By leveraging TF-IDF, FastText, and CNN-based architectures, the system effectively distinguishes between human-written and bot-generated text with an impressive accuracy of 94%. The comparative analysis with state-of-the-art transfer learning models further validates the effectiveness of the proposed approach. The combination of CNN and FastText not only improves detection accuracy but also enhances computational efficiency, making it suitable for real-time applications.

One of the key advantages of this approach is its ability to generalize well across different datasets by capturing complex linguistic patterns and subtle anomalies in synthetic text. The hybrid model incorporating deep learning and traditional machine learning techniques demonstrates improved performance over standalone models, showcasing the benefits of multi-method integration. Moving forward, this research sets a foundation for more advanced studies in deepfake text detection, paving the way for future innovations. The integration of quantum NLP and transfer learning techniques could further enhance detection capabilities, offering even greater accuracy and efficiency. As misinformation continues to evolve, the need for sophisticated detection mechanisms remains critical. By continuously improving and refining these models, this project contributes to the ongoing fight against digital deception, ensuring a more reliable and trustworthy digital communication landscape.

REFERENCES

- [1] T. Fagni, F. Falchi, M. Gambini, et al., “Tweepfake: About detecting deepfake tweets,” *PLOS One*, vol. 16, pp. 1–20, Jan. 2021.
- [2] S. Mitrović, D. Andreoletti, and O. Ayoub, “ChatGPT or human? Detect and explain,” *arXiv preprint*, vol. 2023, pp. 1–10, Jan. 2023.
- [3] R. Vaibhav, R. Furqan, A. Aashir, P. B. Washington, E. Lee, and A. Imran, “Deepfake tweets classification using stacked Bi-LSTM and word embeddings,” *PeerJ Computer Science*, vol. 7, pp. e745, Dec. 2021.
- [4] A. Heidari, N. J. Navimipour, H. Dag, and M. Unal, “Deepfake detection using deep learning methods: A systematic and comprehensive review,” *WIREs Data Mining and Knowledge Discovery*, vol. 14, no. 2, pp. e1520, Mar. 2024.
- [5] J. Ternovski, J. Kalla, and P. M. Aronow, “Deepfake warnings for political videos increase disbelief but do not improve discernment: Evidence from two experiments,” Ph.D. dissertation, *Dept. of Political Science*, Yale University, 2021.
- [6] S. Bradshaw, H. Bailey, and P. N. Howard, “Industrialized disinformation: 2020 global inventory of organized social media manipulation,” *Computational Propaganda Project*, Oxford Internet Institute, University of Oxford, Tech. Rep., 2021.
- [7] X. Liu, Y. Zheng, Z. Du, M. Ding, Y. Qian, Z. Yang, and J. Tang, “GPT understands, too,” *arXiv preprint*, arXiv:2103.10385, 2021.
- [8] L. Beckman, “The inconsistent application of internet regulations and suggestions for the future,” *Nova Law Review*, vol. 46, no. 2, p. 277, 2021, Art. no. 2.
- [9] J.-S. Lee and J. Hsiang, “Patent claim generation by fine-tuning OpenAI GPT-2,” *World Patent Information*, vol. 62, Sep. 2020, Art. no. 101983.

PUBLICATION









